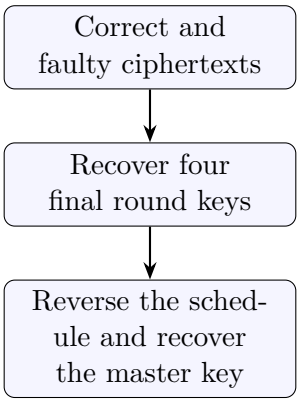
Polytech Montpellier — PFE 5A

Understanding SM4 Fault-Based Key Recovery

A line-by-line illustrated execution guide to

`piret_sm4.py` and `run.py`



Guilherme Martins Specht

Academic supervisor: Prof. Philippe Maurine
Industrial collaboration: STMicroelectronics

Independent educational document — 8 November 2026

Contents

1 What this lesson demonstrates **1**
- 1.1 The question we will answer . . . 1
- 1.2 Run exactly this experiment . . . 1
- 1.3 Separate what the target knows from what the attacker knows . . . 2
- 1.4 A map of the files . . . 2

2 Read the SM4 round before attacking it **3**
- 2.1 Bytes, words and numbering . . . 3
- 2.2 Four calculations form one round . . . 3
- 2.3 Translate the normal terminal line . . . 4
- 2.4 The last clean round of our example . . . 4

3 Follow the Python call chain **5**
- 3.1 Who calls whom? . . . 5
- 3.2 The two nested callbacks in `sm4_demo` . . . 5
- 3.3 What data is stored? . . . 6

4 First fault: recover candidates for byte 0 of rk31 **7**
- 4.1 Step 1: the external callback changes one byte . . . 7
- 4.2 Step 2: the S-box changes the difference . . . 7
- 4.3 Step 3: linear diffusion spreads that byte difference . . . 7
- 4.4 Step 4: keep only the public ciphertexts . . . 8
- 4.5 Step 5: derive the equation that cancels the hidden old word . . . 8

5 Understand inverse L without treating it as magic **9**
- 5.1 What does GF(2) mean here? . . . 9
- 5.2 How `inverse_rows()` constructs the inverse . . . 9
- 5.3 A smaller four-bit drawing of the same method . . . 9
- 5.4 Two equivalent ways the code applies the inverse . . . 10

6 Turn a difference into a recovered key byte **11**
- 6.1 Select the affected byte and compute q . . . 11
- 6.2 First observation: two inputs remain . . . 11
- 6.3 Second observation: intersect, do not guess . . . 12
- 6.4 Understand the recovery loop and Python's for–else . . . 12

7 Recover all four tail keys, not just rk31 **14**
- 7.1 Finish the other bytes of rk31 . . . 14
- 7.2 Why do we need new faults in earlier rounds? . . . 14
- 7.3 How `peel_sm4()` restores an older window . . . 14
- 7.4 A real round-30 example from the execution . . . 15
- 7.5 Result and actual query count . . . 15

8 Reverse the schedule to obtain the original master key **16**
8.1 Public CK and FK are not secret material . . . 16
8.2 Four recovered subkeys seed the inverse recurrence . . . 16
8.3 Read one inverse schedule step . . . 16
8.4 Remove FK and assemble the 128-bit key . . . 17

9 Final checks, logs and presentation checklist **18**
9.1 What does PASS establish? . . . 18
9.2 How the terminal explanation and logs are produced . . . 18
9.3 Common misunderstandings to avoid . . . 19
9.4 A short oral explanation . . . 19

10 Connect the Python statements to a physical experiment **20**
10.1 Three different meanings of "execution" . . . 20
10.2 What happens to a word on a processor? . . . 20
10.3 A timing drawing: trigger, computation and disturbance . . . 21
10.4 Why a physical attempt is not the same as a modeled observation . . . 22
10.5 The candidate set exists on the PC, not inside the chip . . . 22
10.6 Source for the physical-platform concepts . . . 22

11 How to read the function-by-function pages **24**

12 Line-by-line: `piret_sm4.py` **25**
12.1 `sm4_fault_encrypt` . . . 25
12.2 `inverse_rows` . . . 26
12.3 `inverse_linear` . . . 28
12.4 `sm4_byte_candidates` . . . 29
12.5 `reverse_sm4` . . . 31
12.6 `peel_sm4` . . . 32
12.7 `recover_sm4_master_key` . . . 33
12.8 `main` . . . 37

13 Line-by-line: `run.py` **39**
13.1 `block` . . . 39
13.2 `LessonTerminal` . . . 39
13.3 `aes_demo` . . . 41
13.4 `inverse_rows` . . . 45
13.5 `inverse_linear` . . . 46
13.6 `sm4_encrypt` . . . 47
13.7 `sm4_byte_candidates` . . . 47
13.8 `reverse_sm4` . . . 48
13.9 `show_sm4_peeling` . . . 48
13.10 `show_sm4_key_schedule` . . . 50
13.11 `show_sm4_observation` . . . 51
13.12 `show_aes_key_schedule` . . . 55
13.13 `sm4_demo` . . . 56
13.14 `main` . . . 59

A All 32 clean encryption rounds **61**

B All observations used in full recovery **62**

C All inverse-L bit contributions for the first fault **63**

D All 256 candidate inputs for the first two faults **64**

E All 32 inverse key-schedule calculations **69**

F Supporting source excerpts with real line numbers **70**
F.1 Source excerpts: `piret_aes.py` 70
F.2 Source excerpts: `sm4.py` 71
F.3 Source excerpts: `attack_steps.py` 73
F.4 Source excerpts: `terminal_log.py` 78

Chapter 1

What this lesson demonstrates

1.1 The question we will answer

How can a program recover an unknown SM4 master key by comparing correct and faulty ciphertexts? We will follow one complete execution, connect each equation to the Python code, and explain why every recovered byte is justified by observations.

This document is independent of the main AES/SM4 report. It includes its own explanations, drawings, numerical tables and current-code excerpts. All generated numerical tables come from an actual execution of the implementation, not manually invented intermediate values.

Precise scope. The module is named `piret_sm4.py`, but the experiment is a controlled-byte SM4 DFA extension. It is not the AES Piret attack applied unchanged. The model assumes a known XOR mask at a selected S-box input byte in each of rounds 31, 30, 29 and 28. Physical glitch access with these properties has not been demonstrated here.

1.2 Run exactly this experiment

From the repository root:

```
python src/run.py --algorithm sm4 --step
```

The equivalent attack-specific entry point is:

```
python src/piret_sm4.py --step
```

The former calls `run.main()` and then `sm4_demo()`. The latter parses its arguments and calls the same `run.sm4_demo()`. The mathematical lesson is shared; log filenames differ.

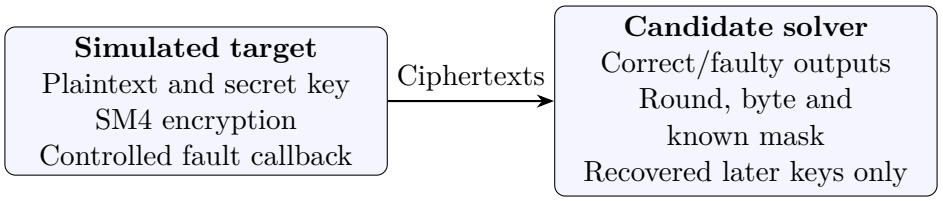
Press Enter at a teaching pause. A stage can contain hundreds of printed candidate tests: scroll within the terminal before continuing. Remove `-step` to execute continuously. The default limit is eight observations *per target byte*; it is not eight faults for the entire recovery.

The worked vector is fixed:

```
Plaintext:  0123456789ABCDEFFEDCBA9876543210
Master key: 0123456789ABCDEFFEDCBA9876543210
Ciphertext: 681EDF34D206965E86B3E94F536E4246
```

Interactive defaults come from `sm4.KEY` and `sm4.PLAINTEXT`. If you change them, the equations still apply but the printed numbers change. The document generator deliberately uses the fixed public vector above.

1.3 Separate what the target knows from what the attacker knows



Internal values may be printed in the teacher view.

No master key, hidden state or reference subkeys are inputs.

`oracle(round, byte, mask)` is a callable that requests a faulty encryption. It represents an experimental target interface, not a cryptographic oracle that reveals the key. In this simulation the callback closes over the target key; its return value is only a ciphertext.

1.4 A map of the files

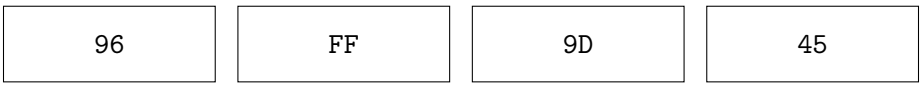
File	Responsibility
`sm4.py`	Normal SM4 encryption, decryption and generic observation hooks.
`piret_sm4.py`	Fault adapter, inverse diffusion, candidate equations, round peeling, recovery and inverse schedule.
`run.py`	Terminal orchestration, comparisons, pauses, observation reporting and final checks.
`attack_steps.py`	Detailed numerical teaching events used by terminal and GUI.
`terminal_log.py`	Mirror execution to the terminal, TXT and colored HTML logs.

Chapter 2

Read the SM4 round before attacking it

2.1 Bytes, words and numbering

One byte is eight bits. One SM4 word is four bytes, or 32 bits. A block contains four words, or 16 bytes. Word byte order is big-endian: byte 0 is the leftmost, most significant byte. Byte and word values in the worked cryptographic calculations are hexadecimal; round numbers and bit indices are decimal.



`96FF9D45`: one word, not four independent states

Round indices are zero-based: round 31 is the *32nd* and final round. SM4 creates words $X_4, \ldots, X_{35}$. The final ciphertext is

$$C = X_{35} \parallel X_{34} \parallel X_{33} \parallel X_{32}.$$

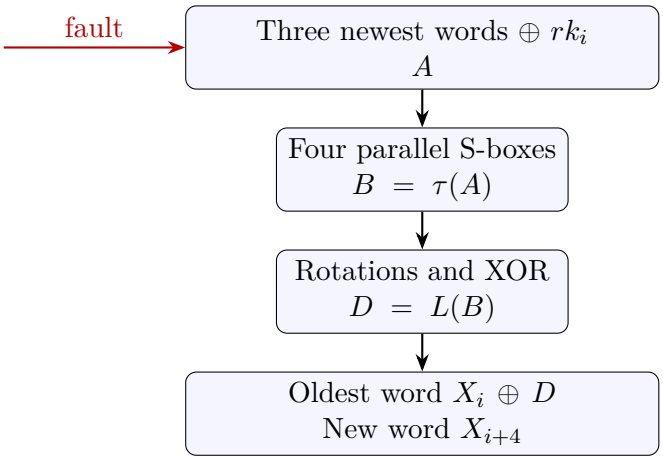
Word reversal is not byte reversal within each word.

2.2 Four calculations form one round

For round i:

$$\begin{aligned} A &= X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i, \\ B &= \tau(A), \\ D &= L(B) = B \oplus \mathrm{rol}(B, 2) \oplus \mathrm{rol}(B, 10) \oplus \mathrm{rol}(B, 18) \oplus \mathrm{rol}(B, 24), \\ X_{i+4} &= X_i \oplus D. \end{aligned}$$

The four bytes pass independently through the SM4 S-box in τ. The rotations in L mix bits across the word. All rotations operate on 32 bits and wrap around.



2.3 Translate the normal terminal line

```
Round 00: A=F002C39E B=18E992B1 L(B)=26D99622 new=27FAD345
```

`run.sm4_demo()` first obtains the clean trace with `sm4_encrypt()`, a wrapper around the target adapter with no selected fault byte. The `for item in trace` loop prints one record per round. A record is a dictionary: `item['a']` is the S-box input; `item['new']` is the new word.

`sm4.round()` calls `observe()` around each intermediate value. Without an observer, a value is returned unchanged. With a callback, returning `None` preserves it; returning an integer replaces it. This separates a normal cipher from attack-specific interventions.

2.4 The last clean round of our example

$$X_{31} = \texttt{7B938F4C}, \qquad rk_{31} = \texttt{9124A012},$$
$$A = \texttt{96FF9D45}, \qquad B = \texttt{1A488C73},$$
$$L(B) = \texttt{138D5078}, \qquad X_{35} = \texttt{681EDF34}.$$

Check the last equation:

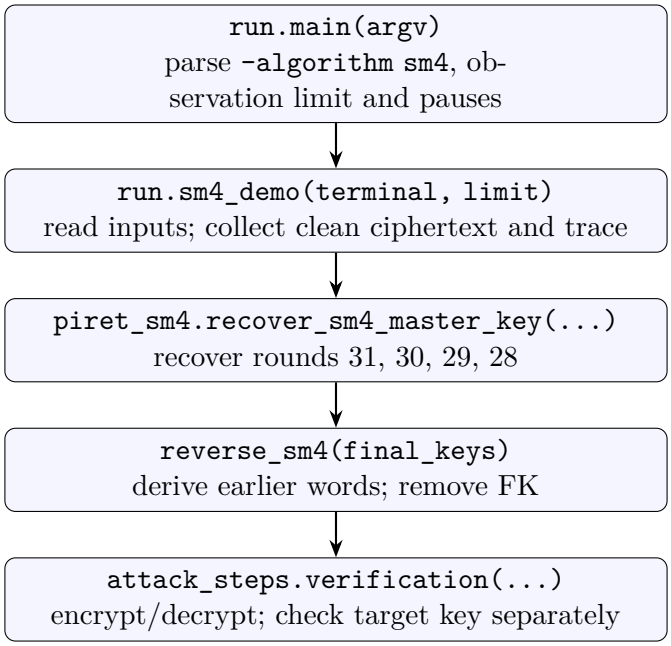
$$\texttt{7B938F4C} \oplus \texttt{138D5078} = \texttt{681EDF34}.$$

These internal values are shown to verify the simulation. The solver must recover rk_{31}, not take it from this display. All 32 clean-round records are reproduced in Appendix A.

Chapter 3

Follow the Python call chain

3.1 Who calls whom?



The detailed presentation may reverse the schedule again to print every numerical step. That repetition is a consistency check of the display, not a second independent key recovery or an additional fault query.

3.2 The two nested callbacks in `sm4_demo`

```
1 def oracle(number, byte, mask):
2     faulty, fault_trace = piret_sm4.sm4_fault_encrypt(plaintext, key, byte, mask, number)
3     # Teacher-only propagation is displayed here.
4     return faulty
5
6 def report(event):
7     show_sm4_observation(terminal, event, recovered_bytes, start_byte)
```

This shortened excerpt omits only the teacher-printing call. The complete current function is included in the source appendix. These are nested functions: they can read the surrounding `plaintext`, `key` and `terminal`. Only the oracle accesses the target key. The report callback receives copied observation metadata after the solver has computed the intersection.

`range(1, limit+1)` supplies masks $01, 02, \ldots$ up to the requested maximum. Although the first two masks each flip one bit, this controlled-byte model permits other nonzero byte masks, including multi-bit masks. The program stops early whenever a byte becomes unique.

3.3 What data is stored?

Variable	Meaning
`keys`	Dictionary of already recovered round keys; initially empty.
`subkey`	Four-byte buffer for the current round key.
`survivors`	Set of compatible key-byte values. `None` means no observation yet.
`fresh`	Candidates compatible with only the current ciphertext pair.
`history`	Observation dictionaries used by logs and GUI explanations.
`used`	Number of actual faulty ciphertext queries.
`clean_view`	Ciphertext-style four-word view restored to the target round.
`faulty_view`	Equivalent restored view for the faulty encryption.

Chapter 4

First fault: recover candidates for byte 0 of rk31

4.1 Step 1: the external callback changes one byte

The relevant line in `sm4_fault_encrypt()` is:

```
1 if number == fault_round and operation == "Sbox input" and byte is not None:
2     value ^= mask << (24 - 8 * byte)
```

The three conditions select the round, intermediate value and whether a fault was requested. `byte=None` creates a clean run. For `byte=0`, the shift is 24, so mask `01` becomes `01000000`. The augmented assignment `^=` XORs the mask into the current 32-bit value.

$$\texttt{96FF9D45} \oplus \texttt{01000000} = \texttt{97FF9D45}.$$

The observer returns the changed value to the actual SM4 round. Its trace stores `a`, `b`, `linear` and `new` for teaching.

4.2 Step 2: the S-box changes the difference

For the selected input byte:

$$S(\texttt{96}) = \texttt{1A}, \qquad S(\texttt{97}) = \texttt{55}.$$

Therefore

$$\Delta B = \texttt{1A488C73} \oplus \texttt{55488C73} = \texttt{4F000000}.$$

$$\boxed{96 \oplus 97 = 01} \xrightarrow{\text{S-box}} \boxed{1A \oplus 55 = 4F}$$

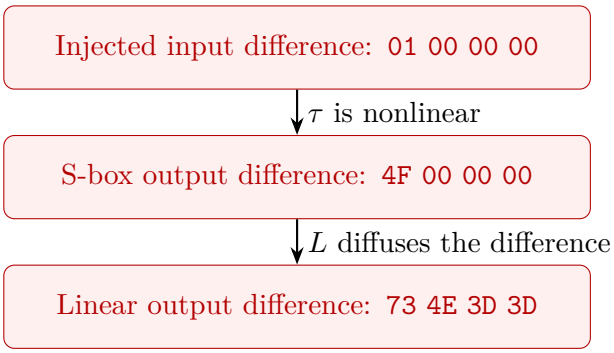
One changed input byte remains one changed S-box output byte.
The nonlinear S-box changes its difference from 01 to $4F$.

4.3 Step 3: linear diffusion spreads that byte difference

$$L(\texttt{4F000000}) = \texttt{734E3D3D}.$$

The four bytes are now nonzero. Since the oldest input word remains unchanged, this is also the newest output-word difference:

$$X'_{35} = \texttt{681EDF34} \oplus \texttt{734E3D3D} = \texttt{1B50E209}.$$



Why does the teacher-view XOR contain many bytes? Its four displayed words are A, $\tau(A)$, $L(\tau(A))$, and the new word. They are four intermediate results concatenated for display, not the complete cipher state. Only the first injected difference is a one-byte mask.

4.4 Step 4: keep only the public ciphertexts

```
Correct: 681EDF34 D206965E 86B3E94F 536E4246
Faulty:  1B50E209 D206965E 86B3E94F 536E4246
XOR:     734E3D3D 00000000 00000000 00000000
```

`struct.unpack(">4I", ciphertext)` converts 16 bytes into four big-endian unsigned words. The solver checks `c[1:] == f[1:]`. If any of those three unchanged words differs, this pair does not satisfy the assumed restored last-round model and is rejected.

4.5 Step 5: derive the equation that cancels the hidden old word

The two last-round equations are:

$$X_{35} = X_{31} \oplus L(B), \qquad X'_{35} = X_{31} \oplus L(B').$$

XORing them cancels X_{31} because a value XORed with itself is zero:

$$\begin{aligned} \Delta X &= X_{35} \oplus X'_{35} \\ &= X_{31} \oplus X_{31} \oplus L(B) \oplus L(B') \\ &= L(B \oplus B') = L(\Delta B). \end{aligned}$$

So the attacker can compute $\Delta B = L^{-1}(\Delta X)$ without knowing X_{31}, B, B' or the round key.

Chapter 5

Understand inverse L without treating it as magic

5.1 What does GF(2) mean here?

GF(2) contains only 0 and 1. Addition is XOR and multiplication is AND. In particular $1 + 1 = 0$. A 32-bit word can be treated as a vector of 32 such coefficients.

Rotations rearrange bit positions. XOR combines them. Thus $L(u \oplus v) = L(u) \oplus L(v)$, and L can be represented by a binary matrix M:

$$\Delta X = M\Delta B, \qquad \Delta B = M^{-1}\Delta X.$$

Here arithmetic is bitwise GF(2), not the AES byte-field multiplication in GF(2^8).

5.2 How `inverse_rows()` constructs the inverse

1. For each bit position i, compute `sm4.linear(1 << i)`. This gives column i of M.
2. Assemble 32 rows and append an identity matrix: $[M \mid I]$.
3. For each column, select a pivot row containing a 1 and swap it into place.
4. XOR that row into every other row having a 1 in the same column.
5. The left half becomes I, and the right half becomes M^{-1}.

There is no division: the only nonzero pivot in GF(2) is 1. The implementation packs each augmented row into a Python integer; the lower 32 bits store M and the upper 32 bits store I. After elimination, `row >> 32` extracts an inverse row. `@lru_cache(maxsize=1)` caches this public, key-independent matrix.

5.3 A smaller four-bit drawing of the same method

This is a toy matrix, *not* the SM4 matrix, so we can see the entire operation:

$$\left[\begin{array}{cccc|cccc} 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{array}\right] \longrightarrow \left[\begin{array}{cccc|cccc} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{array}\right].$$

The toy matrix maps 1010 to 1110; the right-hand inverse maps 1110 back to 1010. The real code performs the same kind of elimination with 32 rows.

5.4 Two equivalent ways the code applies the inverse

The solver's expanded `inverse_linear(value)` calculates a dot product for each inverse row in explicit steps:

```
selected_bits = row & value
set_bit_count = bin(selected_bits).count("1")
parity = set_bit_count & 1
result |= parity << index
```

`row & value` selects shared set bits. Counting them and keeping the lowest bit computes parity: an even count gives 0, an odd count gives 1. Shifting that parity to its output position and ORing disjoint bit positions assembles the word.

The detailed terminal instead explains the equivalent column/basis expansion:

$$L^{-1}(v) = \bigoplus_{i:v_i=1} L^{-1}(2^i).$$

It prints all 32 input-bit contributions. `inverse basis` is $L^{-1}(2^i)$; `contribution` is that value if the input bit is 1, or zero otherwise. `running XOR` is the accumulated answer. Bit 0 is the rightmost, least significant bit.

For our example the first three relevant calculations are:

$$\begin{aligned} i = 0: &\quad 0 \oplus \texttt{41455115} = \texttt{41455115}, \\ i = 1: &\quad \texttt{41455115} \oplus 0 = \texttt{41455115}, \\ i = 2: &\quad \texttt{41455115} \oplus \texttt{05154455} = \texttt{44501540}. \end{aligned}$$

After all bits:

$$\boxed{L^{-1}(\texttt{734E3D3D}) = \texttt{4F000000}}.$$

The terminal checks $L(\texttt{4F000000}) = \texttt{734E3D3D}$. Appendix C prints every contribution.

Chapter 6

Turn a difference into a recovered key byte

6.1 Select the affected byte and compute q

For byte 0, `shift = 24 - 8*byte` equals 24. Right shifting ΔB by 24 and masking with 255 selects $d = \texttt{4F}$. The code rejects differences with nonzero bits outside that byte.

The S-box input is

$$A = X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}.$$

The three data words are visible in the ciphertext. Their XOR is:

$$\texttt{D206965E} \oplus \texttt{86B3E94F} \oplus \texttt{536E4246} = \texttt{07DB3D57}.$$

Its first byte is $q = \texttt{07}$. If the unknown first S-box input byte is a, then

$$a = q \oplus k, \qquad \boxed{k = a \oplus q}.$$

The solver guesses the unknown *S-box input* a first. It converts compatible inputs into key-byte guesses using q. The variable `inputs` is therefore not the same set as the returned key-byte candidates.

6.2 First observation: two inputs remain

For known mask $e = \texttt{01}$, try every $a = 00, \ldots, FF$:

$$S(a) \oplus S(a \oplus \texttt{01}) = \texttt{4F}.$$

Only $a = \texttt{96}$ and $a = \texttt{97}$ pass. For instance:

$$S(\texttt{96}) \oplus S(\texttt{97}) = \texttt{1A} \oplus \texttt{55} = \texttt{4F}.$$

The order can be reversed and the XOR stays the same, explaining this pair of inputs. Convert them to key guesses:

$$\texttt{96} \oplus \texttt{07} = \texttt{91}, \qquad \texttt{97} \oplus \texttt{07} = \texttt{90}.$$

So the first key-byte set is $\{\texttt{90}, \texttt{91}\}$. Even though the teacher view showed input 96, the solver must not use that hidden value to discard 90.

6.3 Second observation: intersect, do not guess

Request a new fault at the *same* byte with mask 02. The faulty ciphertext is

```
ED9DD932 D206965E 86B3E94F 536E4246
```

The recovered S-box difference is `81000000`, giving $d = \texttt{81}$. The compatible inputs are:

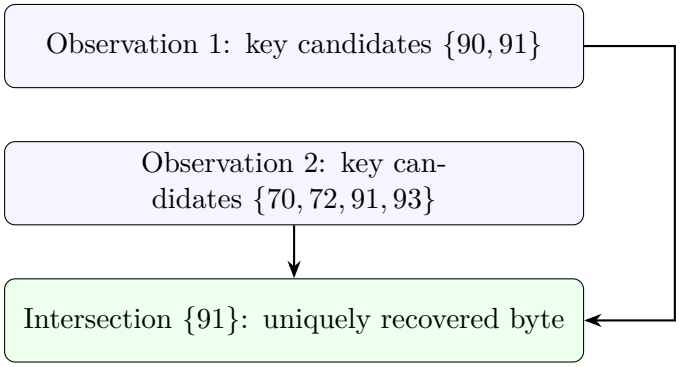
$$\{\texttt{75}, \texttt{77}, \texttt{94}, \texttt{96}\}.$$

After XOR with $q = 07$, the fresh key guesses are:

$$\{\texttt{70}, \texttt{72}, \texttt{91}, \texttt{93}\}.$$

Intersect with the first observation:

$$\boxed{\{90, 91\} \cap \{70, 72, 91, 93\} = \{91\}}.$$



The terminal prints `RECOVERED rk31 byte 0 = 91 / UNIQUE`. Full input tests for both observations are in Appendix D.

6.4 Understand the recovery loop and Python's for–else

The essential blocks in `recover_sm4_master_key()` are:

```
survivors = None
for observation, mask in enumerate(masks, 1):
    faulty = bytes(fault_oracle(number, byte, mask))
    faulty_view = peel_sm4(faulty, number, keys)
    fresh, inputs, delta = sm4_byte_candidates(clean_view, faulty_view, byte, mask)
    if survivors is None:
        survivors = set(fresh)
    else:
        survivors = survivors & fresh
    if not survivors:
        raise ValueError(...)
    if len(survivors) == 1:
        subkey[byte] = next(iter(survivors))
        break
else:
    return ...  # incomplete result, with master_key=None
```

This is an explanatory excerpt: event creation and the optional alternative filter are omitted here but preserved in the full source appendix.

- `enumerate(masks,1)` gives a human-readable observation count starting at 1.
- `None` means no constraint yet; an empty set means inconsistent observations.
- `&` intersects sets. It does not perform byte XOR.

- `next(iter(survivors))` is safe here only because the length was checked to be exactly one.
- `break` stops querying this byte once it is unique.
- The `else` belongs to the *for loop*, not the preceding if: it runs only if the loop ends without a break. An ambiguous byte blocks full recovery.

Chapter 7

Recover all four tail keys, not just rk31

7.1 Finish the other bytes of rk31

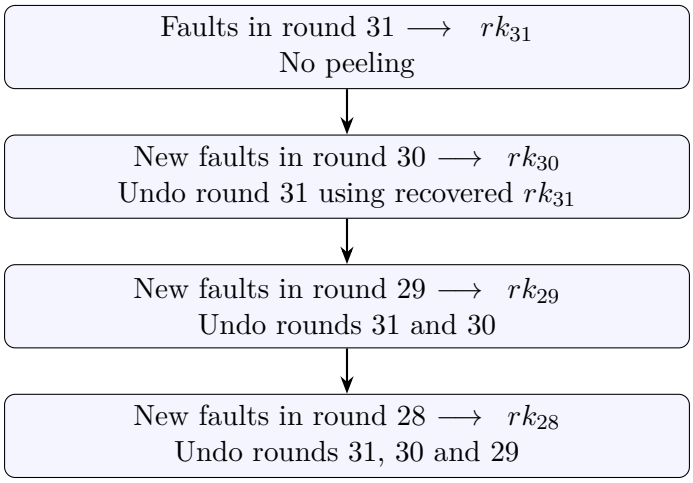
The same equations recover bytes 1, 2 and 3. Their q bytes are $DB, 3D, 57$, respectively. They become $24, A0, 12$. Concatenate in big-endian order:

$$rk_{31} = \texttt{91} \parallel \texttt{24} \parallel \texttt{A0} \parallel \texttt{12} = \texttt{9124A012}.$$

`int.from_bytes(subkey,"big")` performs this assembly. Four unique bytes are required before `keys[31]` is populated.

7.2 Why do we need new faults in earlier rounds?

One 32-bit subkey does not supply the four consecutive words required to invert SM4's 128-bit key schedule. Recover the final four subkeys in reverse chronological order:

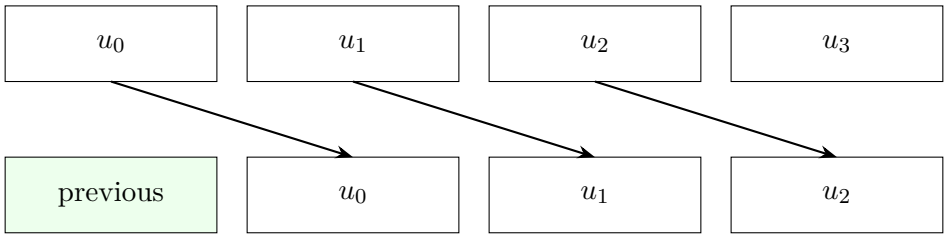


The program does not reuse a last-round faulty ciphertext and pretend it reveals earlier keys. It requests a new target encryption at each selected earlier round.

7.3 How `peel_sm4()` restores an older window

Let the chronological post-round state be (u_0, u_1, u_2, u_3). With the recovered key of that round:

$$\begin{aligned} A &= u_0 \oplus u_1 \oplus u_2 \oplus rk, \\ \text{previous} &= u_3 \oplus L(\tau(A)), \\ \text{restored state} &= (\text{previous}, u_0, u_1, u_2). \end{aligned}$$



The newest word and recovered key reconstruct the missing oldest word.

`words = unpack(...)[::-1]` restores chronological order. The loop runs only over later rounds. A missing recovered key raises an error. The function finally serializes `words[::-1]` again to preserve the ciphertext-style layout expected by the byte filter.

7.4 A real round-30 example from the execution

Below, `correct`/`faulty` are actual final ciphertexts. `clean_view`/`faulty_view` are synthetic restored outputs after undoing round 31. They are not a second physical measurement.

```
FIRST ROUND-30 OBSERVATION (new fault, not reused round-31 data)
correct: 681EDF34D206965E86B3E94F536E4246
faulty: AE21A70A772712DA86B3E94F536E4246
clean_view: D206965E86B3E94F536E42467B938F4C
faulty_view: 772712DA86B3E94F536E42467B938F4C
CORRECT
Chronological window: 536E4246 86B3E94F D206965E 681EDF34
A=96FF9D45; tau(A)=1A488C73; L(tau(A))=138D5078
Restored older word=681EDF34 XOR 138D5078=7B938F4C
FAULTY
Chronological window: 536E4246 86B3E94F 772712DA AE21A70A
A=33DE19C1; tau(A)=A9B4441B; L(tau(A))=D5B22846
Restored older word=AE21A70A XOR D5B22846=7B938F4C
```

After peeling, the same newest-word difference equation and unchanged-word checks apply to round 30. Repeat the process for rounds 29 and 28.

7.5 Result and actual query count

For this fixed example every byte becomes unique after two observations:

$$4 \text{ target rounds} \times 4 \text{ bytes} \times 2 \text{ observations} = 32 \text{ faulty ciphertexts.}$$

There is also a clean reference ciphertext. Diagnostic replays and schedule printing do not increase `result['used']`.

Recovered key	Value
rk_{31}	`9124A012`
rk_{30}	`01CF72E5`
rk_{29}	`62293496`
rk_{28}	`428D3654`

The count 32 is the measured result for this vector and these masks, not a universal fault requirement. Appendix B lists all 32 observations, fresh candidate sets and intersections.

Chapter 8

Reverse the schedule to obtain the original master key

8.1 Public CK and FK are not secret material

The four fixed FK words are

$$\texttt{A3B1BAC6}, \quad \texttt{56AA3350}, \quad \texttt{677D9197}, \quad \texttt{B27022DC}.$$

They initialize $K_i = MK_i \oplus FK_i$. The 32 CK words are generated from public byte values:

$$ck_{i,j} = 7(4i + j) \bmod 256, \qquad CK_i = ck_{i,0} \parallel \cdots \parallel ck_{i,3}.$$

For example $CK_0 = \texttt{00070E15}$, $CK_{31} = \texttt{646B7279}$. FK values are directly specified constants; the implementation does not derive them from CK. Neither set depends on the secret key.

8.2 Four recovered subkeys seed the inverse recurrence

$$K_{32} = rk_{28}, \quad K_{33} = rk_{29}, \quad K_{34} = rk_{30}, \quad K_{35} = rk_{31}.$$

The forward recurrence is

$$K_{i+4} = K_i \oplus L'(\tau(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)).$$

XOR is self-inverse, so

$$\boxed{K_i = K_{i+4} \oplus L'(\tau(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i))}.$$

The code iterates `range(31,-1,-1)`: 31, 30, ..., 0. This order guarantees that the four required higher-index words are already known.

8.3 Read one inverse schedule step

For $i = 31$, calculate:

$$U = K_{32} \oplus K_{33} \oplus K_{34} \oplus CK_{31}.$$

Then substitute all four bytes and apply

$$L'(B) = B \oplus \mathrm{rol}(B, 13) \oplus \mathrm{rol}(B, 23).$$

Finally compute $K_{31} = K_{35} \oplus L'(\tau(U)) = \texttt{F1780C81}$.

Do not confuse the transforms. Encryption uses rotations 2, 10, 18 and 24. Key expansion uses 13 and 23. The code correctly calls `sm4.linear_key()`, not `sm4.linear()`, inside `reverse_sm4()`.

The terminal prints every S-box lookup and rotation. Earlier encryption keys $rk_{27}, \ldots, rk_0$ are labeled `DERIVED`: they come from the schedule, not additional fault observations. Appendix E contains all 32 numerical inverse steps.

8.4 Remove FK and assemble the 128-bit key

The first four recovered schedule words still contain whitening:

$$MK_i = K_i \oplus FK_i \quad (i = 0, 1, 2, 3).$$

i	K_i	FK_i	$MK_i = K_i \oplus FK_i$	Role
0	`A292FFA1`	`A3B1BAC6`	`01234567`	Master word
1	`DF01FEBF`	`56AA3350`	`89ABCDEF`	Master word
2	`99A12B0F`	`677D9197`	`FEDCBA98`	Master word
3	`C42410CC`	`B27022DC`	`76543210`	Master word

Concatenate the four 32-bit words:

$$\boxed{MK = \texttt{01234567} \parallel \texttt{89ABCDEF} \parallel \texttt{FEDCBA98} \parallel \texttt{76543210}}.$$

`struct.pack(">4I", ...)` produces the original 16 key bytes in big-endian word order.

Chapter 9

Final checks, logs and presentation checklist

9.1 What does PASS establish?

The terminal independently verifies:

1. Encryption with the recovered key equals the original correct ciphertext.
2. Decryption of the correct ciphertext with the recovered key equals the plaintext.
3. The displayed inverse schedule agrees with the recovery engine.
4. The recovered key equals the simulated target key, revealed for ground-truth validation.

These checks establish consistency for the executed software experiment. A correct round trip by itself would not prove that an attack had recovered the target key; the ciphertext and ground-truth comparisons matter.

```
Recovered key: 0123456789ABCDEFFEDCBA9876543210
Encrypt(P, recovered key): 681EDF34D206965E86B3E94F536E4246
Decrypt(C, recovered key): 0123456789ABCDEFFEDCBA9876543210
SM4 MASTER KEY: 0123456789ABCDEFFEDCBA9876543210 / PASS
32 faulty outputs. All four subkeys came from DFA, not references.
```

This block combines the final values for readability; exact heading wording is determined by the executable version.

9.2 How the terminal explanation and logs are produced

`show_sm4_observation()` calls the detailed event generator in `attack_steps.sm4_observation()`. It explains peeling, all inverse-L contributions and all 256 candidate inputs, then prints concise recovered-byte summaries. Seeing a summary after a detailed table is not an extra observation.

`LessonTerminal.pair()` prints correct/faulty/XOR values and colors changed bytes red. A nonzero XOR means a difference, not necessarily an additional fault. `terminal.pause()` only waits when `-step` was requested; it does not change any cryptographic values.

`run_with_log()` surrounds the complete entry point. The Tee objects duplicate stdout and stderr. The TXT file preserves ANSI sequences; a final HTML conversion renders red differences and green recovery results. A new execution overwrites only the same program's previous log.

Command entry point	Log pair
`run.py`	`logs/run.txt`, `logs/run.html`
`piret_sm4.py`	`logs/piret_sm4.txt`, `logs/piret_sm4.html`

9.3 Common misunderstandings to avoid

- A one-byte injected error can spread across a whole word after L.
- $\Delta B = 4F000000$ is an S-box output difference, not the mask 01 and not the key.
- q is calculated from observable unchanged words, not read from a hidden register.
- Multiple candidates mean ambiguity. They are not interchangeable recovered keys.
- One recovered round key is insufficient to reverse the complete SM4 schedule.
- Earlier-round peeling uses recovered later keys; reference keys are not solver inputs.
- This software target accepts exact round/byte/mask requests. A physical clock or voltage glitch does not automatically provide that control.

9.4 A short oral explanation

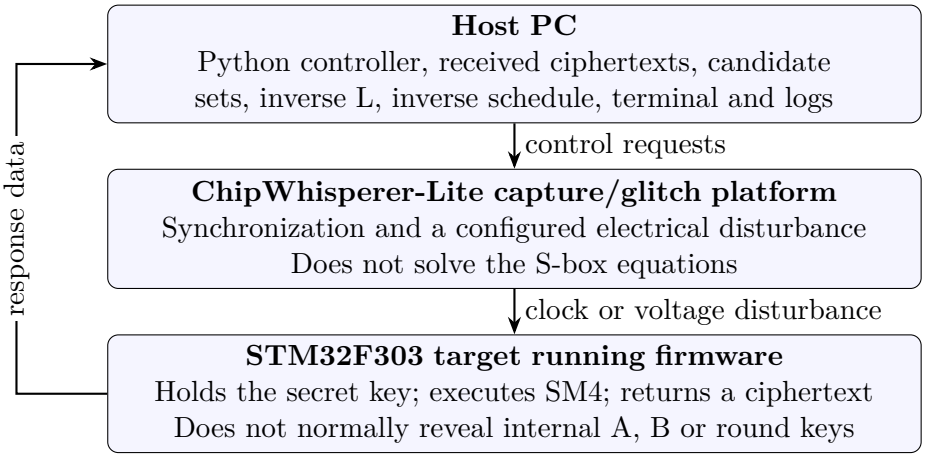
I first run normal SM4 to obtain a reference ciphertext. I inject a known byte XOR at a selected final-round S-box input. The ciphertext XOR removes unchanged terms. Inverting the public linear transformation isolates an S-box output difference. I enumerate possible S-box inputs, convert them into key-byte guesses using public words, and intersect independent observations until one value remains. After recovering rk31, I request new faults in earlier rounds and undo only rounds whose keys are already recovered. Four consecutive recovered tail keys let me reverse the key schedule, remove FK and reconstruct the master key. Finally I verify encryption, decryption and simulation ground truth.

Chapter 10

Connect the Python statements to a physical experiment

10.1 Three different meanings of "execution"

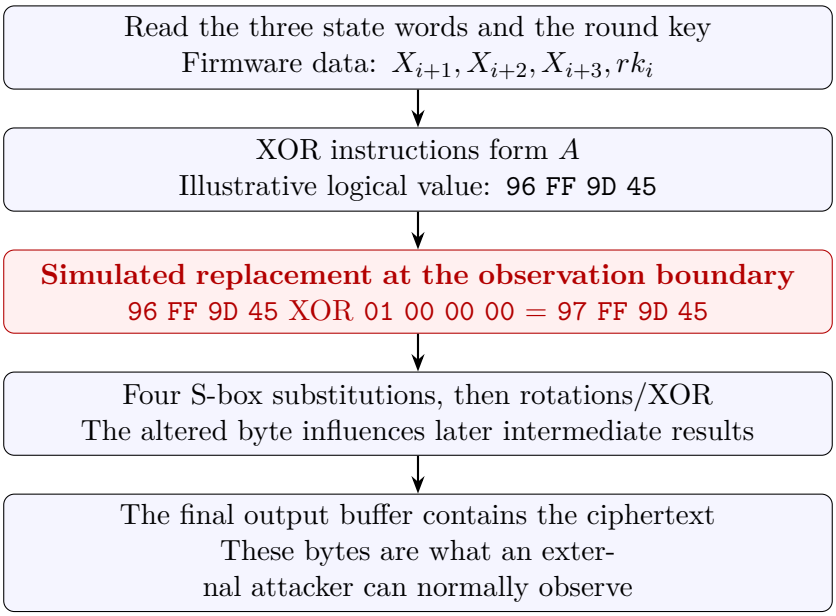
Today: all Python functions in this lesson run on your PC. The simulator calls `sm4.py`; it does not communicate with the ChipWhisperer or STM32. The following drawings describe how those roles would map to a future board experiment. They are conceptual diagrams, not measured waveforms or a wiring guide.



The response path is schematic: an actual transport implementation determines how the host exchanges commands and bytes with the target. The current `oracle()` is a Python callable, not that transport. Clock and voltage glitching are distinct configurations; this drawing does not imply both are applied together.

10.2 What happens to a word on a processor?

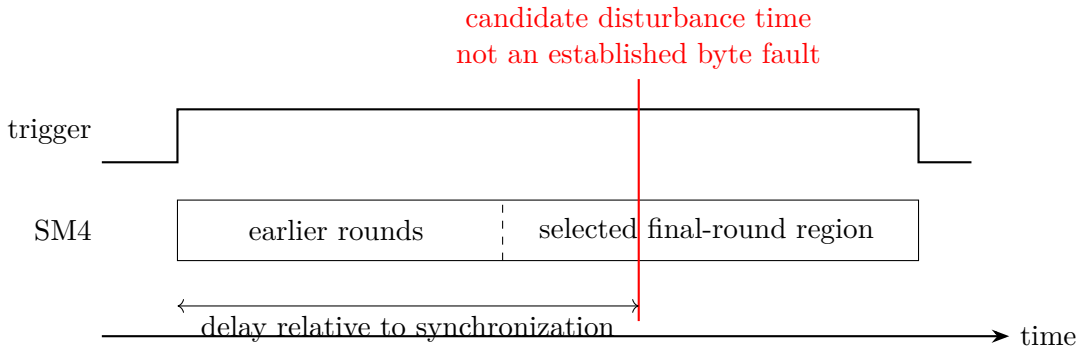
In Python, `value` is an integer object in the host process. In compiled firmware, a corresponding intermediate word may be in a CPU register, stack or RAM, depending on the compiler and instructions. Without examining the firmware binary and timing, we cannot assign it to a specific physical register or clock edge.



The red box is exactly what `value ^= mask << (24 - 8 * byte)` models. A physical glitch does *not* execute this Python assignment and does not inherently guarantee mask 01 or this precise intermediate boundary.

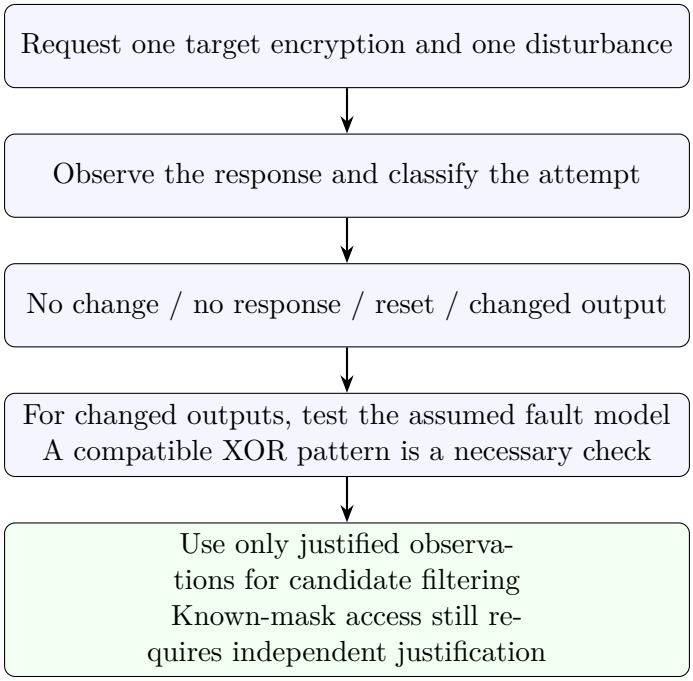
10.3 A timing drawing: trigger, computation and disturbance

The horizontal scale below is symbolic; it is not an experimentally measured timing window.



ChipWhisperer documentation describes trigger-synchronized glitch generation and routing for clock or voltage experiments. Whether the disturbance affects the intended operation must be characterized on the actual target. A trigger near encryption gives a time reference, not automatic knowledge of a round, byte or XOR mask.

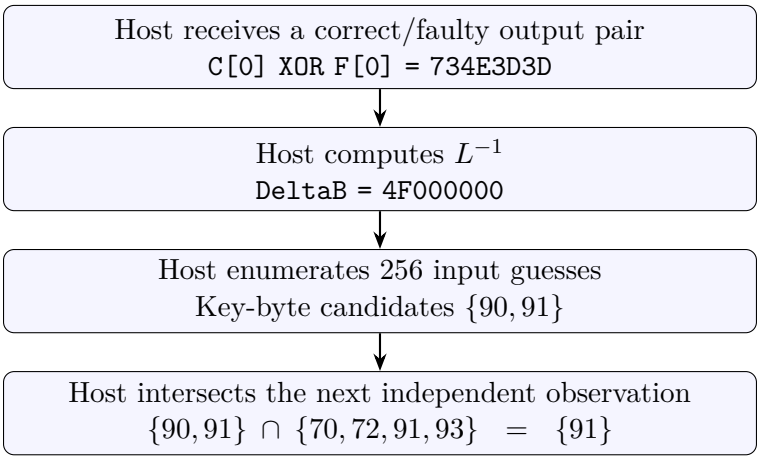
10.4 Why a physical attempt is not the same as a modeled observation



The software model directly produces the selected byte XOR, so its `used=32` counts successful modeled ciphertext queries. A physical campaign must also count ineffective attempts, resets, rejected faults and acquisition failures. Its total attempt count is not established by the software value 32.

An output pattern consistent with a model does not by itself prove the precise physical fault location or known mask. Do not use the secret test key to select observations and then claim the attacker knew the mask independently.

10.5 The candidate set exists on the PC, not inside the chip



No operation in this drawing rewinds the target, reads its secret memory or removes bits electrically. The host is solving equations about bytes it already received.

10.6 Source for the physical-platform concepts

The hardware-role and synchronization discussion follows NewAE's official training material, Fault 1.1: Introduction to Clock Glitching and Fault 2.1: Introduction to Voltage Glitching. Those

tutorials illustrate platform concepts; their example target behavior does not validate this SM4 byte-fault model on an STM32F303.

Chapter 11

How to read the function-by-function pages

The following two chapters cover *every function defined in* ***piret_sm4.py*** *and* ***run.py***, including nested callbacks and methods defined in the latter's terminal class. The AES-only functions in `run.py` are explicitly labeled as inactive for `-algorithm sm4`; they are included so no function in either requested file is silently skipped.

For each function, read in this order:

1. The purpose sentence and its input–processing–output diagram.
2. The physical-interpretation box, which says what happens on the PC and what only models target behavior.
3. The complete numbered source, copied from the current repository.
4. The line table, explaining each executable physical source line and every continuation or alternate branch.

Blank lines, comments and docstrings remain visible in the numbered source but have no separate execution step; the table does not pretend they perform computations. A function definition creates a callable, and a nested callback body runs later only when called. Loops cause a source line to execute repeatedly; they are not new source lines on each iteration.

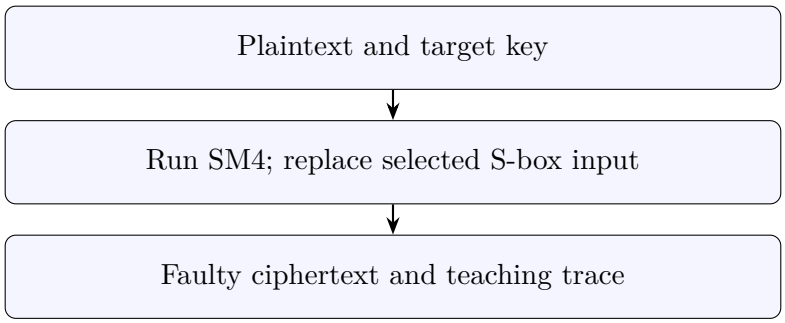
The line tables describe what each statement does, not one CPU instruction per Python line. A Python expression can execute many host operations; compiled STM32 firmware has a different instruction stream. Current source line numbers and the table's coverage manifest are refreshed whenever this PDF is rebuilt.

Chapter 12

Line-by-line: `piret_sm4.py`

12.1 `sm4_fault_encrypt`

Simulate a controlled fault during the real SM4 implementation, while collecting teacher-only intermediates.



Physical interpretation: Target-side behavior is modeled here. In firmware, intermediate words could occupy CPU registers or RAM depending on compilation; this Python callback does not access STM32 registers.

Exact source

```
14 def sm4_fault_encrypt(plaintext, key, byte=None, mask=0, fault_round=31):
15     """Run sm4.py with an external S-box-input XOR in a selected round.
16 
17     This PDF model assumes a controlled known byte fault. Giraud restricts the
18     mask to a single bit before reusing this target adapter.
19     """
20 
21     if byte is not None and not (0 <= byte < 4 and 1 <= mask <= 255):
22         raise ValueError("SM4 fault requires byte 0..3 and mask 1..255.")
23 
24     if not 0 <= fault_round < 32:
25         raise ValueError("SM4 fault round must be 0..31.")
26 
27     # A list holds one 32-bit round key at each index from 0 through 31.
28     keys = sm4.key_expansion(key)
29 
30     # {round: i, rk: 32-bit integer}
31     trace = []
32 
33     for index in range(32):
34         trace.append(dict(round=index, rk=keys[index]))
35 
36     fields = {"Sbox input": "a", "Sbox output": "b", "Linear output": "linear", "New word": "new"}
37 
38     def observer(number, operation, value):
39         # Byte 0 is the most significant byte: move its mask to bits 31..24.
40         # Returning this replacement changes the real cipher at this exact event.
41         if number == fault_round and operation == "Sbox input" and byte is not None:
```

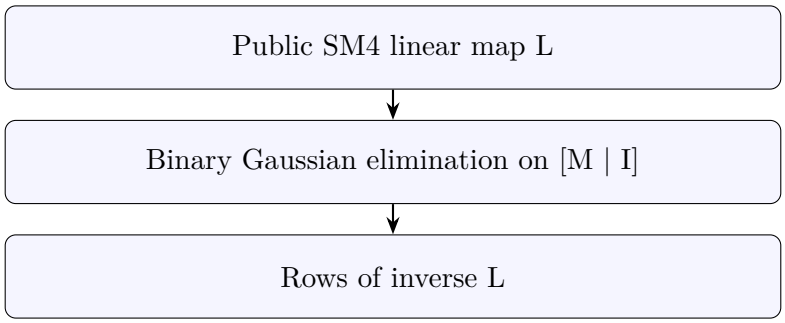
```
42              value ^= mask << (24 - 8 * byte)
43
44          if operation in fields:
45              trace[number][fields[operation]] = value
46
47          return value
48
49      ciphertext = sm4.sm4_encrypt_block(plaintext, key, observer=observer)
50
51      return ciphertext, trace
```

Read each executable line

Line	Meaning and effect
14	Declare this callable and its parameters; the body runs only when called. Simulate a controlled fault during the real SM4 implementation, while collecting teacher-only intermediates.
21	If a fault was requested, require a byte position 0..3 and a nonzero one-byte mask. None permits a clean reference run.
22	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
24	Reject a selected round outside the zero-based 0..31 interval.
25	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
28	Compute the target schedule for trace labeling. The result is a list indexed 0..31, not a dictionary; it is not passed to the output-only solver.
31	Create an empty teaching trace; the following explicit loop adds one record for each round.
33	Visit all thirty-two round indices to construct the trace records.
34	Add the record for this round and its target-side round key to the teacher trace.
36	Translate cipher callback event names into short trace dictionary fields a, b, linear and new.
38	Define the target callback; its body executes later when the normal cipher calls the observation hook, not when this definition is reached.
41	Inject only when both round and operation match, and a target byte was selected; other events pass through unchanged.
42	Move the mask into the selected big-endian byte and XOR it into the word. For byte 0 and mask 01, the shifted mask is 01000000.
44	Record only named scalar intermediate events; ignore other observation events for this teacher trace.
45	Save the observed scalar at its round and semantic field, so the terminal can compare clean and faulty calculations.
47	Return the unchanged or injected scalar to the normal SM4 engine; this returned replacement is what the next cipher operation consumes.
49	Execute normal SM4 with the external observer attached. A returned observer replacement is consumed by the real round function.
51	Return the target output together with teacher-only intermediate records. The oracle later forwards only the ciphertext to the solver.

12.2 `inverse_rows`

Construct and cache the inverse of the public 32-bit diffusion map.



Physical interpretation: This is attacker-side algebra on the PC. No pulse, board read or secret key is required.

Exact source

```
@lru_cache(maxsize=1)
def inverse_rows():
    """Invert SM4 L's 32x32 binary matrix with Gaussian elimination."""

    # L(2^i) gives column i of L's binary matrix: each input bit is a basis vector.
    columns = []

    for bit_index in range(32):
        basis_word = 1 << bit_index

        transformed_word = sm4.linear(basis_word)

        columns.append(transformed_word)

    # Store each augmented row [L | I] in one 64-bit integer; upper bits hold I.
    rows = []

    for row_index in range(32):
        matrix_row = 0

        for column_index in range(32):
            output_bit = (columns[column_index] >> row_index) & 1
            matrix_row |= output_bit << column_index

        identity_bit = 1 << (32 + row_index)

        augmented_row = matrix_row | identity_bit

        rows.append(augmented_row)

    for column in range(32):
        # Find a row with a 1 in this column, then move it to the pivot position.
        pivot = None

        for candidate_row in range(column, 32):
            pivot_bit = (rows[candidate_row] >> column) & 1

            if pivot_bit == 1:
                pivot = candidate_row
                break

        if pivot is None:
            raise StopIteration()

        rows[column], rows[pivot] = rows[pivot], rows[column]

        for row in range(32):
            if row != column and (rows[row] >> column) & 1:
                # XOR eliminates the column bit; GF(2) requires no division.
                rows[row] ^= rows[column]

    # The lower half is now I; the upper half contains the rows of inverse L.
    inverse_matrix_rows = []

```

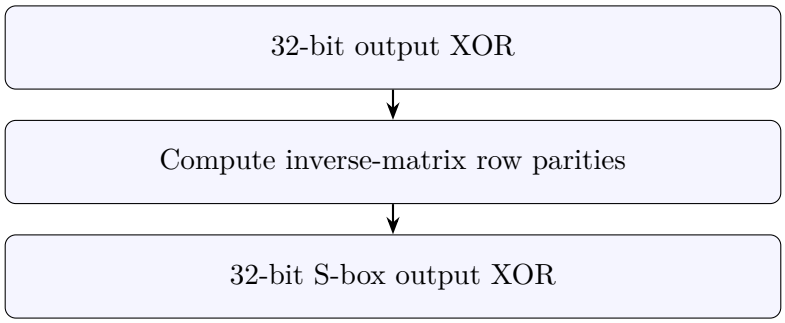
```
108     for augmented_row in rows:
109         inverse_row = augmented_row >> 32
110 
111         inverse_matrix_rows.append(inverse_row)
112 
113     return tuple(inverse_matrix_rows)
```

Read each executable line

Line	Meaning and effect
54	Cache the public inverse-matrix result after the first calculation. The matrix is key-independent and is not recomputed for every observation.
55	Declare this callable and its parameters; the body runs only when called. Construct and cache the inverse of the public 32-bit diffusion map.
59	Prepare the list of images of the thirty-two input basis words under the public map L.
61	Choose each input basis bit explicitly, from bit zero through bit thirty-one.
62	Construct a 32-bit word with exactly the selected input bit set.
64	Apply public encryption diffusion L to that basis word, obtaining one matrix column.
66	Store this basis image as the next column of the binary matrix.
69	Prepare the augmented-matrix row list before encoding individual row bits.
71	Construct each output row of the binary matrix explicitly.
72	Start this encoded matrix row with every bit cleared.
74	Read the selected output bit from each stored matrix column.
75	Extract the bit indicating whether this input column contributes to the selected output row.
76	Place that contribution bit in the encoded matrix row; OR combines disjoint bit positions.
78	Place the matching identity-matrix bit in the upper half of the augmented row.
80	Combine the L-matrix row with its identity bit to form the packed row of [M | I].
82	Store the completed augmented row before Gaussian elimination.
84	Process one pivot column of the 32-by-32 binary matrix at a time.
86	Mark the pivot as not yet found; the following ordinary loop searches eligible rows.
88	Search the current and remaining rows for a one in the pivot column.
89	Read the current candidate row bit at the pivot column.
91	Accept this row as a valid pivot only if its selected column bit is one.
92	Remember the actual row index, then stop searching because a valid pivot was found.
93	Stop searching pivot rows as soon as a valid row has been found.
95	Detect a missing pivot instead of blindly swapping an invalid row index.
96	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
98	Swap the chosen pivot row into the current diagonal position.
100	Visit every augmented row so the current column can be cleared outside its pivot.
101	Leave the pivot unchanged; eliminate only another row whose pivot-column bit is one.
103	XOR the entire augmented pivot row into that row, performing GF(2) row elimination on both halves.
106	Prepare the final public inverse-matrix rows after elimination.
108	Visit each reduced augmented row to extract its inverse half.
109	Discard the lower identity half and retain the upper inverse-matrix row.
111	Add the extracted inverse row in the original output-bit order.
113	Return an immutable sequence of inverse rows, cached for reuse by later observations.

12.3 `inverse_linear`

Apply the inverse binary matrix to one observed output difference.



Physical interpretation: The PC transforms received ciphertext bytes into a mathematical difference. The target does not run backward.

Exact source

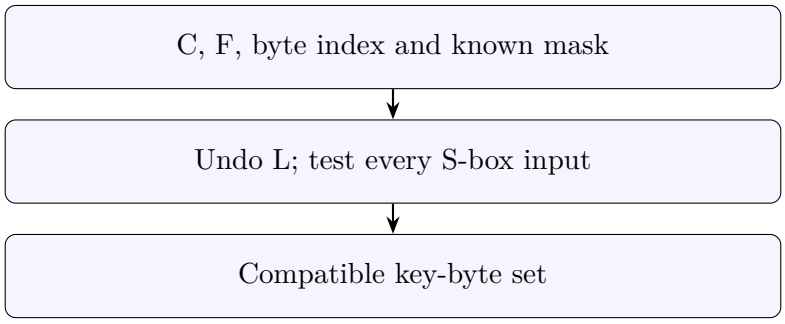
```
116 def inverse_linear(value):
117     # AND selects input bits used by each inverse-matrix row. Their parity is
118     # the GF(2) dot product; shift that one-bit result into its output position.
119     result = 0
120 
121     for index, row in enumerate(inverse_rows()):
122         selected_bits = row & value
123 
124         set_bit_count = bin(selected_bits).count("1")
125 
126         parity = set_bit_count & 1
127 
128         result |= parity << index
129 
130     return result
```

Read each executable line

Line	Meaning and effect
116	Declare this callable and its parameters; the body runs only when called. Apply the inverse binary matrix to one observed output difference.
119	Clear the output word before placing one parity bit at a time.
121	Associate each inverse-matrix row with the output bit it computes.
122	Select exactly the input bits used by this inverse row.
124	Count selected one bits; only whether this count is odd or even matters over GF(2).
126	Keep the lowest count bit: odd becomes one and even becomes zero.
128	Place this row parity into its output position without changing other output bits.
130	Return the word assembled from all thirty-two inverse-row parity calculations.

12.4 `sm4_byte_candidates`

Find key-byte guesses consistent with one restored ciphertext pair and a known byte fault.



Physical interpretation: This filtering happens on the PC after a response has been received. The known-mask assumption must be established separately for a physical experiment.

Exact source

```
133 def sm4_byte_candidates(correct, faulty, byte, mask):
134     """Attacker API: outputs and the PDF's KNOWN fault byte/value only."""
135 
136     if len(correct) != 16 or len(faulty) != 16 or not 0 <= byte < 4 or not 1 <= mask <= 255:
137         raise ValueError("Invalid SM4 output or known-fault descriptor.")
138 
139     c, f = struct.unpack(">4I", correct), struct.unpack(">4I", faulty)
140 
141     # The restored last-round model allows only its newest output word to change.
142     if c[1:] != f[1:]:
143         raise ValueError("The last-round model requires three output words unchanged.")
144 
145     # Cancel the unchanged older word and undo diffusion to isolate S-box differences.
146     delta_b = inverse_linear(c[0] ^ f[0])
147 
148     shift = 24 - 8 * byte
149 
150     if not delta_b or delta_b & ~(255 << shift):
151         # Reject both a missing fault and differences outside the selected byte.
152         raise ValueError("Difference does not match a single-byte S-box input fault.")
153 
154     d = (delta_b >> shift) & 255
155 
156     # These three output words are public; their selected XOR byte is q.
157     known = ((c[1] ^ c[2] ^ c[3]) >> shift) & 255
158 
159     # Enumerate unknown S-box inputs a, then convert each to a key guess a XOR q.
160     inputs = set()
161 
162     for candidate_input in range(256):
163         faulty_input = candidate_input ^ mask
164 
165         output_difference = sm4.SBOX[candidate_input] ^ sm4.SBOX[faulty_input]
166 
167         if output_difference == d:
168             inputs.add(candidate_input)
169 
170     key_candidates = set()
171 
172     for candidate_input in inputs:
173         key_byte = candidate_input ^ known
174 
175         key_candidates.add(key_byte)
176 
177     return key_candidates, inputs, delta_b
```

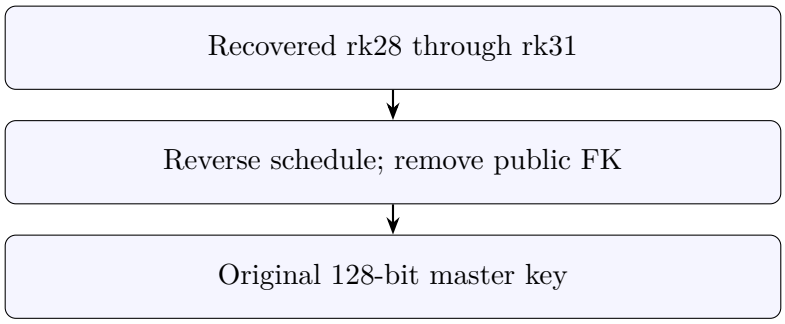
Read each executable line

Line	Meaning and effect
133	Declare this callable and its parameters; the body runs only when called. Find key-byte guesses consistent with one restored ciphertext pair and a known byte fault.
136	Require two complete block outputs and valid controlled-fault descriptors before interpreting differences.
137	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
139	Decode each sixteen-byte output into four big-endian unsigned 32-bit words.
142	Require the last three restored output words to be unchanged. Otherwise this observation violates the selected last-round model.
143	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
146	XOR the newest words, then undo public linear diffusion to recover the S-box output difference.

Line	Meaning and effect
148	Map byte 0,1,2,3 to bit shifts 24,16,8,0 in a big-endian 32-bit word.
150	Reject a zero difference or any nonzero difference bit outside the selected byte.
152	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
154	Select the affected S-box output difference byte, keeping only its lowest eight bits.
157	Compute q: select a byte of the XOR of the three observable unchanged words. It is not a hidden state read.
160	Start an empty set of compatible unknown S-box input bytes.
162	Test each possible unknown input byte explicitly.
163	Apply the known error hypothesis to the candidate S-box input.
165	Calculate the clean/faulty S-box XOR for this candidate input and known error.
167	Keep the hypothesis only if its predicted S-box difference equals the observed one.
168	Record this compatible input value; the set removes duplicates.
170	Prepare key-byte candidates separately from S-box input hypotheses.
172	Convert each compatible S-box input into a key-byte guess.
173	Solve a = q XOR key_byte using the observable q byte.
175	Store the resulting key-byte hypothesis for intersection with other observations.
177	Return the key hypotheses, compatible S-box inputs and full inverse-L difference separately.

12.5 `reverse_sm4`

Derive the initial key words from four consecutive recovered tail words, then remove FK.



Physical interpretation: All schedule inversion runs on the PC. Public CK/FK constants are not extracted from the board.

Exact source

```
180 def reverse_sm4(final_keys, observer=None):
181     """Four consecutive keys rk28..rk31 are REQUIRED, not merely rk31."""
182 
183     if len(final_keys) != 4:
184         raise ValueError("SM4 reversal requires four consecutive final subkeys.")
185 
186     # rk28..rk31 occupy K32..K35, supplying the four-word seed for inversion.
187     words = [None] * 32 + list(final_keys)
188 
189     for index in range(31, -1, -1):
190         # Descending order ensures all words on the right are already known.
191         # CK is public; use the key-schedule transform L', not encryption's L.
192         value = words[index+1] ^ words[index+2] ^ words[index+3] ^ sm4.CK[index]
193 
194         words[index] = words[index+4] ^ sm4.linear_key(sm4.sbox(value))
195 
196         if observer:
197             observer(index, words[index])
198 
199     # K0..K3 still include FK whitening; remove it and serialize the master key.
200     master_words = []
```

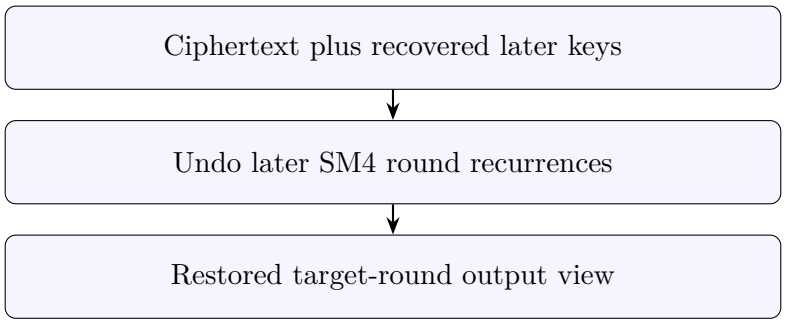
```
201
202     for word_index in range(4):
203         master_word = words[word_index] ^ sm4.FK[word_index]
204         master_words.append(master_word)
205
206     return struct.pack(">4I", *master_words)
```

Read each executable line

Line	Meaning and effect
180	Declare this callable and its parameters; the body runs only when called. Derive the initial key words from four consecutive recovered tail words, then remove FK.
183	Require four consecutive tail subkeys; a single recovered rk31 is insufficient to seed the inverse schedule.
184	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
187	Reserve K0..K31 as unknown and seed K32..K35 with recovered rk28..rk31 in that order.
189	Recover schedule words in descending order so every right-hand dependency is already known.
192	Form the public schedule-core input from three known neighboring words and CK[index].
194	Apply the forward S-box and key-specific L-prime, then XOR with K[index+4] to solve for the older schedule word.
196	Call optional explanatory instrumentation only if supplied; the numerical result has already been computed.
197	Report this derived schedule word without giving the observer control over key recovery.
200	Prepare four unwhitened master-key words.
202	Remove FK separately for master-word positions zero through three.
203	Undo initial key whitening with the corresponding public FK constant.
204	Preserve the master-word order for final serialization.
206	Serialize the four recovered master words into sixteen big-endian bytes.

12.6 `peel_sm4`

Reconstruct an earlier output window using only already recovered later keys.



Physical interpretation: This is offline PC computation. It neither rewinds the STM32 nor reads an old state from target RAM.

Exact source

```
209 def peel_sm4(ciphertext, target_round, recovered_keys):
210     """Undo only rounds whose subkeys have ALREADY been recovered by DFA.
211
212     Return a reversed four-word output at the end of target_round. This lets
213     the same last-round equations analyze rounds 30, 29 and 28 in succession.
214     No reference round keys or secret intermediate values are accepted.
215     """
216
```

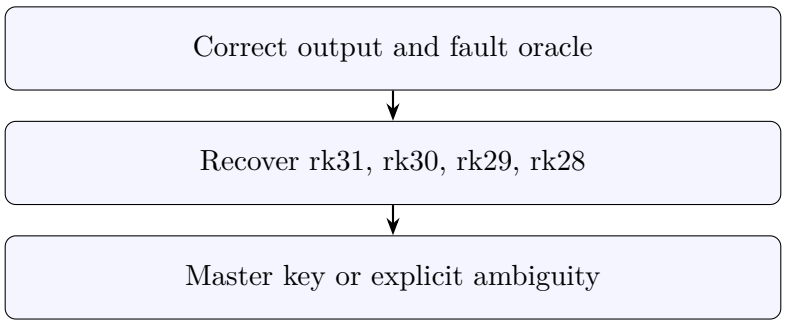
```
217     aes.validate_block(ciphertext, "SM4 ciphertext")
218
219     if not 28 <= target_round <= 31:
220         raise ValueError("Full SM4 recovery targets rounds 28..31.")
221
222     # Ciphertext reverses the final four words; restore chronological state order.
223     words = list(struct.unpack(">4I", ciphertext))[::-1]
224
225     for number in range(31, target_round, -1):
226         if number not in recovered_keys:
227             raise ValueError(f"Cannot undo round {number} before recovering its subkey.")
228
229         # Recompute the round transform with an already recovered key; XOR the
230         # newest word to recover the word that was dropped from the window.
231         previous = words[3] ^ sm4.linear(sm4.sbox(words[0] ^ words[1] ^ words[2] ^ recovered_keys[number]))
232
233         words = [previous, words[0], words[1], words[2]]
234
235     # Return ciphertext-style order so the same last-round filter can be reused.
236     return struct.pack(">4I", *words[::-1])
```

Read each executable line

Line	Meaning and effect
209	Declare this callable and its parameters; the body runs only when called. Reconstruct an earlier output window using only already recovered later keys.
217	Reuse a sixteen-byte length check. This does not execute AES and does not import an AES key into SM4 recovery.
219	Restrict this full-recovery peeling helper to the four modeled target rounds.
220	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
223	Read the four final output words, then reverse their order to restore the chronological rolling state.
225	Undo only rounds later than the requested target round, newest first.
226	Require a genuinely recovered later key before reversing its round; never substitute a reference key.
227	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
231	Recompute the later-round nonlinear/linear transform and XOR with its newest word to reconstruct the missing oldest word.
233	Shift the reconstructed window backward by one round: old, u0, u1, u2.
236	Return a synthetic ciphertext-style view with word reversal, allowing reuse of the same last-round candidate equation.

12.7 `recover_sm4_master_key`

Coordinate independent fault observations, intersections, four tail-key recoveries and inverse key expansion.



Physical interpretation: On hardware, each oracle request would involve another target encryption and response. The implementation here makes software calls, not USB or serial requests.

Exact source

```
239 def recover_sm4_master_key(correct, fault_oracle, masks=(1, 2, 3, 4, 5, 6, 7, 8), start_byte=0,
    ↪ candidate_filter=None, observer=None):
240     """Recover rk31 -> rk28 and reverse the schedule, using outputs only.
241 
242     fault_oracle(round, byte, mask) returns a faulty ciphertext. It represents
243     a target capable of a controlled fault in EACH of the final four rounds.
244     The solver receives no master key, schedule, or reference subkeys.
245     The default filter uses known masks; Giraud supplies its single-bit filter.
246     """
247 
248     aes.validate_block(correct, "Correct SM4 ciphertext")
249 
250     masks = tuple(masks)
251 
252     if not masks:
253         raise ValueError("Use distinct nonzero byte masks and start_byte 0..3.")
254 
255     if len(set(masks)) != len(masks):
256         raise ValueError("Use distinct nonzero byte masks and start_byte 0..3.")
257 
258     for candidate_mask in masks:
259         if not 1 <= candidate_mask <= 255:
260             raise ValueError("Use distinct nonzero byte masks and start_byte 0..3.")
261 
262     if not 0 <= start_byte < 4:
263         raise ValueError("Use distinct nonzero byte masks and start_byte 0..3.")
264 
265     keys, history, counts, used = {}, [], {}, 0
266 
267     for number in range(31, 27, -1):
268         # Recover later keys first; only those keys may be used for round peeling.
269         clean_view = peel_sm4(correct, number, keys)
270 
271         subkey = bytearray(4)
272 
273         byte_order = []
274 
275         for offset in range(4):
276             byte_order.append((start_byte + offset) % 4)
277 
278         for byte in byte_order:
279             survivors = None
280 
281             # None means no observation yet, not an empty or failed candidate set.
282             for observation, mask in enumerate(masks, 1):
283                 faulty = bytes(fault_oracle(number, byte, mask))
284                 # The oracle knows the target secret, but returns only ciphertext.
285                 faulty_view = peel_sm4(faulty, number, keys)
286 
287                 if candidate_filter is None:
288                     # Default DFA knows the mask; Giraud supplies a different filter.
289                     fresh, inputs, delta = sm4_byte_candidates(clean_view, faulty_view, byte, mask)
290                 else:
291                     position, fresh, inputs, delta = candidate_filter(clean_view, faulty_view, mask)
292 
293                     if position != byte:
294                         raise ValueError("Fault observation does not match the requested byte.")
295 
296                 if survivors is None:
297                     before = 256
298                     survivors = set(fresh)
299                 else:
300                     before = len(survivors)
301                     survivors = survivors & fresh
302 
303                 # Independent observations can remove candidates, never add them.
304                 if not survivors:
305                     raise ValueError("SM4 candidate intersection is empty; fault model is incompatible.")
306 
307                 used += 1
308                 counts[f"{number}:{byte}"] = len(survivors)
309                 recovered_key_labels = {}
310 
```

```
311                 for index, value in keys.items():
312                     recovered_key_labels[str(index)] = f'{value:08X}'
313
314                 event = dict(round=number, byte=byte, observation=observation, mask=mask, before=before,
                    ↪  pair_candidates=len(fresh), remaining=len(survivors), survivors=sorted(survivors),
                    ↪  inputs=sorted(inputs), delta_b=delta, correct=correct.hex(), faulty=faulty.hex(),
                    ↪  clean_view=clean_view.hex(), faulty_view=faulty_view.hex(),
                    ↪  peeled_with=recovered_key_labels)
315                 history.append(event)
316
317                 if observer is not None:
318                     # Copy event metadata so presentation code cannot edit solver state.
319                     observer(dict(event))
320
321                 if len(survivors) == 1:
322                     # Only uniqueness permits using this byte in a recovered round key.
323                     subkey[byte] = next(iter(survivors))
324                     break
325             else:
326                 # The loop exhausted its budget without a unique byte; do not peel
327                 # an earlier round with a guessed key or claim a complete master key.
328                 return dict(master_key=None, round_keys=keys, history=history, candidate_counts=counts,
                    ↪  used=used, complete=False, blocked_round=number)
329
330         keys[number] = int.from_bytes(subkey, "big")
331
332     # All four tail keys now come from observations; invert the schedule once.
333     final_keys = []
334
335     for candidate_number in range(28, 32):
336         final_keys.append(keys[candidate_number])
337
338     master = reverse_sm4(final_keys)
339
340     return dict(master_key=master, round_keys=keys, history=history, candidate_counts=counts, used=used,
    ↪  complete=True)
```

Read each executable line

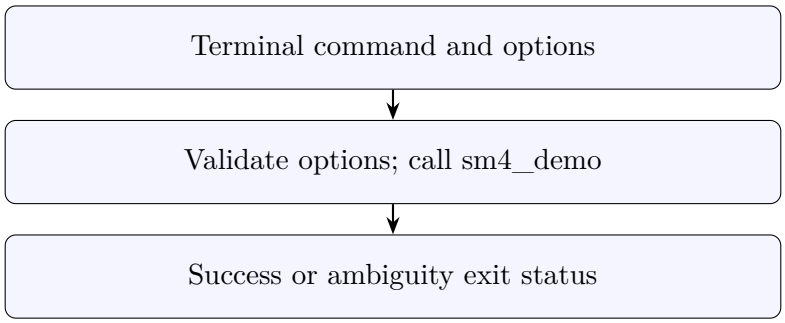
Line	Meaning and effect
239	Declare this callable and its parameters; the body runs only when called. Coordinate independent fault observations, intersections, four tail-key recoveries and inverse key expansion.
248	Reuse a sixteen-byte length check. This does not execute AES and does not import an AES key into SM4 recovery.
250	Freeze a possibly one-shot iterable so every target byte can reuse the same ordered observation masks.
252	Reject an empty fault-mask sequence before any target query.
253	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
255	Reject repeated masks; deterministic repeated observations do not add constraints.
256	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
258	Validate each supplied fault descriptor explicitly.
259	Reject zero and masks that do not fit in one nonzero byte.
260	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
262	Require the first selected byte position to be zero through three.
263	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
265	Initialize recovered keys, event history, per-byte counts and actual faulty-output query count separately.
267	Target rounds 31,30,29,28 in that order; later recovered keys enable peeling for the next target.
269	Restore the clean reference to this target round using only the currently recovered later keys.

Line	Meaning and effect
271	Allocate four output bytes for the current round key; unfilled bytes must not be treated as recovered.
273	Prepare an explicit ordering of all four byte positions for this round.
275	Visit four offsets starting from the user-selected first byte.
276	Wrap the byte position modulo four so every position appears once.
278	Recover one complete key byte at a time in the chosen order.
279	Mark the byte or group as unconstrained before its first observation; an empty set would instead mean inconsistency.
282	Request distinct faults sequentially, numbering observations from one within this target byte/group.
283	Query the simulated target for one new faulty ciphertext. No key or teacher trace is returned to this solver.
285	Undo the same recovered later rounds in the faulty ciphertext so both outputs refer to the same target round.
287	Choose the default known-byte filter unless another compatible filter, such as the Giraud-inspired one, was explicitly supplied.
289	Apply the selected-round byte equation to this clean/faulty pair and known mask.
290	The if-condition at line 287 was false; follow this alternative path rather than the preceding branch.
291	Run the alternate filter and receive its inferred byte position alongside candidate sets and difference.
293	Reject an alternate-filter result whose changed byte is inconsistent with the requested target byte.
294	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
296	Distinguish the first observation from subsequent intersections; no previous candidate set exists yet.
297	Before a first byte observation all 256 byte values are still possible.
298	The first pair initializes the byte set; subsequent pairs intersect it. No new guess can be introduced by intersection.
299	The if-condition at line 296 was false; follow this alternative path rather than the preceding branch.
300	Record the number of hypotheses remaining before applying the new observation.
301	Intersect the prior hypotheses with those from the new pair; never introduce a new candidate.
304	An empty intersection means the observations and model cannot be reconciled; do not claim a key.
305	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
307	Count one faulty ciphertext actually consumed by the solver, not a printed stage or a schedule replay.
308	Store the surviving count for this particular round and byte.
309	Prepare a display-only dictionary of already recovered later subkeys.
311	Visit only subkeys already established by the solver.
312	Format each recovered subkey as eight hexadecimal digits for event metadata.
314	Package the measurement pair, restored views, known mask, input guesses, surviving key guesses and recovered peeling keys for presentation.
315	Preserve this observation in order for later inspection and reproducibility.
317	Only invoke the optional reporting callback if the caller requested it.
319	Pass copied event metadata to reporting. Candidate-set intersection has already been decided in the solver.
321	Uniqueness is the acceptance rule: one value remains compatible with all observations for this byte/group.
323	Extract the sole surviving byte and store it at the correct big-endian position.
324	Stop this observation loop immediately after uniqueness; do not query unused masks.
325	The for-loop beginning at line 282 exhausted its observation budget without break; execute the following ambiguity result.

Line	Meaning and effect
328	Return an explicit incomplete result when the observation loop exhausts its budget without a break. No master key is selected.
330	Concatenate the four uniquely recovered bytes into one 32-bit round key and make it available for future peeling.
333	Prepare the four recovered tail keys in forward key-schedule order.
335	Read rk28, rk29, rk30 and rk31 in the order expected by schedule inversion.
336	Add the actual recovered subkey to the inverse-schedule seed.
338	Arrange recovered rk28..rk31 in schedule order and reverse expansion to derive the original key.
340	Return the complete key together with actual recovered tail keys, history, counts and fault-query cost.

12.8 `main`

Parse the attack-specific command and enter the same SM4 terminal lesson used by run.py.



Physical interpretation: This is a PC entry point. Selecting –row chooses a simulated byte, not a physical chip coordinate. The compatibility options –column and –seed are parsed but not used in this SM4 path.

Exact source

```
343 def main(argv=None):
344     """Use the existing SM4 terminal presentation without changing its output."""
345 
346     parser = argparse.ArgumentParser(description=__doc__)
347 
348     parser.add_argument("--algorithm", choices=("sm4",), default="sm4")
349     parser.add_argument("--step", action="store_true")
350     parser.add_argument("--color", choices=("auto", "always", "never"), default="auto")
351     parser.add_argument("--faults", type=int, default=8, help="Maximum observations per byte, 1..255.")
352     parser.add_argument("--row", type=int, choices=range(4), default=0, help="First target byte.")
353     # Keep the same optional terminal settings for both Piret entry points.
354     parser.add_argument("--column", type=int, choices=range(4), default=0)
355     parser.add_argument("--seed", type=int, default=42)
356 
357     args = parser.parse_args(argv)
358 
359     if not 1 <= args.faults <= 255:
360         parser.error("--faults must be 1..255.")
361 
362     import run
363 
364     if run.sm4_demo(run.LessonTerminal(args.color, args.step), args.faults, args.row):
365         return 0
366 
367     return 2
```

Read each executable line

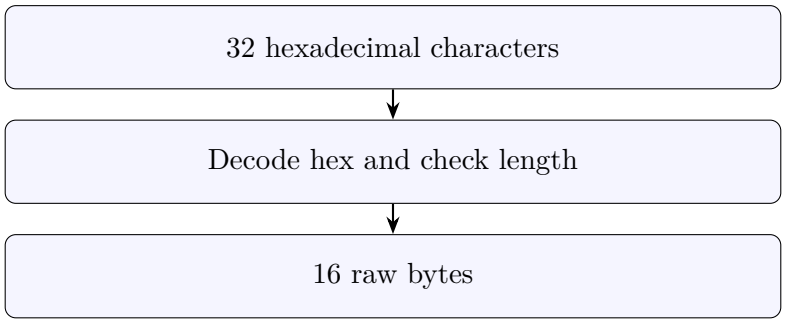
Line	Meaning and effect
343	Declare this callable and its parameters; the body runs only when called. Parse the attack-specific command and enter the same SM4 terminal lesson used by run.py.
346	Create the command-line option parser using the module description for help text.
348	Register –algorithm. Select the allowed cipher branch for this entry point.
349	Register –step. Enable pauses at teaching boundaries.
350	Register –color. Select automatic, forced or disabled ANSI colors.
351	Register –faults. Set the maximum distinct mask observations per SM4 target byte.
352	Register –row. Choose the first simulated SM4 byte position.
354	Register –column. Compatibility setting parsed but not used by this SM4 entry point.
355	Register –seed. Used for AES mask sampling in run.py; not used for SM4 mask order.
357	Decode the supplied argument list (or process command line) and enforce the declared argument types and choices.
359	Reject an observation budget outside 1..255 before requesting any faulty outputs.
360	Report invalid configuration and terminate argument processing rather than starting an invalid experiment.
362	Load the shared terminal controller at call time. This avoids a top-level circular import between attack and presentation modules.
364	Run the shared SM4 lesson and test whether full recovery verified.
365	Return successful process status after verified complete recovery.
367	Return the explicit ambiguity/failure status when full recovery was not verified.

Chapter 13

Line-by-line: `run.py`

13.1 `block`

Convert a hexadecimal input string to exactly sixteen bytes.



Physical interpretation: A future host transport would send these bytes to firmware. This function only prepares host memory.

Exact source

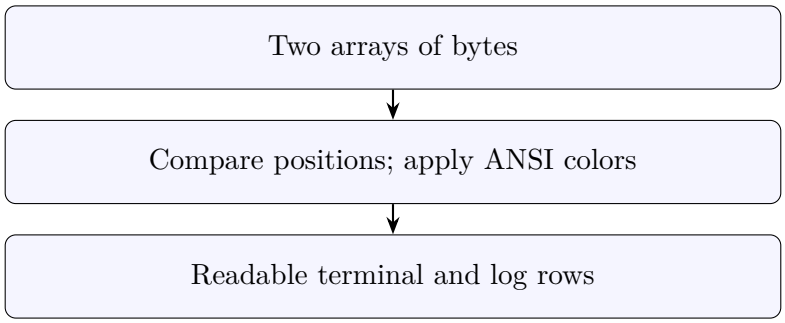
```
12 def block(value):
13     data = bytes.fromhex(value)
14
15     if len(data) != 16:
16         raise ValueError("Key and plaintext must contain exactly 16 bytes as hex.")
17
18     return data
```

Read each executable line

Line	Meaning and effect
12	Declare this callable and its parameters; the body runs only when called. Convert a hexadecimal input string to exactly sixteen bytes.
13	Decode hexadecimal text into raw bytes; malformed hex raises ValueError instead of silently changing the input.
15	Accept only one complete 128-bit block or key for this demonstration.
16	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
18	Return the validated raw bytes used by the cipher and target simulator.

13.2 `LessonTerminal`

Format the same data as clean/faulty/XOR rows with red differences.



Physical interpretation: Only the PC display changes. Red text is a visualization of differing data, not a measured electrical pulse.

Exact source

```
21 class LessonTerminal(piret_aes.Terminal):
22     def changed(self, original, faulty):
23         formatted_values = []
24 
25         for index, value in enumerate(faulty):
26             if value != original[index]:
27                 element_value = self.paint(f'{value:02X}', '31')
28             else:
29                 element_value = f'{value:02X}'
30 
31             formatted_values.append(element_value)
32 
33         return ' '.join(formatted_values)
34 
35     def pair(self, correct, faulty):
36         formatted_values = []
37 
38         for byte_2 in correct:
39             formatted_values.append(f'{byte_2:02X}')
40 
41         print('Correct: ' + ' '.join(formatted_values))
42         print("Faulty:  " + self.changed(correct, faulty))
43 
44         byte_values = []
45 
46         for input_value, byte in zip(correct, faulty):
47             byte_values.append(input_value ^ byte)
48 
49         delta = bytes(byte_values)
50 
51         print("XOR:     " + self.changed(bytes(len(delta)), delta))
```

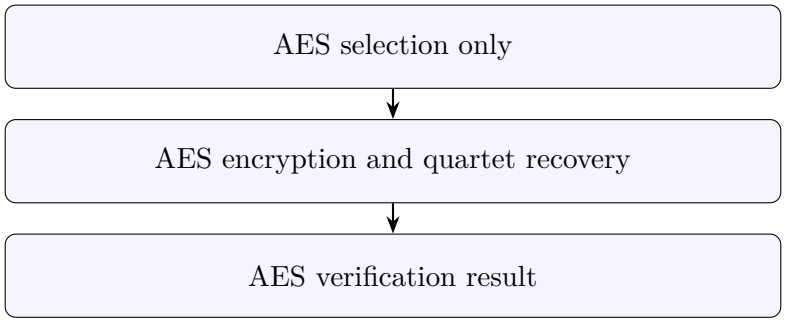
Read each executable line

Line	Meaning and effect
21	Declare this callable and its parameters; the body runs only when called. Format the same data as clean/faulty/XOR rows with red differences.
22	Define the byte-formatting method used to color differences without changing any byte values.
23	Accumulate or append the explicitly formatted hexadecimal display fragments; this does not modify cipher data.
25	Compare each faulty output byte with its reference position before choosing its color.
26	Use red formatting only when this faulty byte differs from the clean reference byte.
27	Prepare this changed byte with ANSI red and an immediate reset.
28	The if-condition at line 26 was false; follow this alternative path rather than the preceding branch.
29	Prepare an unchanged byte as ordinary two-digit hexadecimal text.

Line	Meaning and effect
31	Accumulate or append the explicitly formatted hexadecimal display fragments; this does not modify cipher data.
33	Join the prepared hex fragments with spaces for terminal display.
35	Define the clean/faulty/XOR printing method used throughout both lessons.
36	Accumulate or append the explicitly formatted hexadecimal display fragments; this does not modify cipher data.
38	Format each clean reference byte without fault coloring.
39	Accumulate or append the explicitly formatted hexadecimal display fragments; this does not modify cipher data.
41	Print the text/label "Correct:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
42	Print the text/label "Faulty:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
44	Prepare an explicit byte buffer for the following comparison or recovered-key serialization.
46	Visit the aligned correct and faulty bytes in the same serialized order.
47	Append their byte XOR, which is zero exactly when the two positions agree.
49	Serialize the explicitly constructed difference buffer.
51	Print the text/label "XOR:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.

13.3 `aes_demo`

Run the AES round-9 lesson when AES is selected; this function is not executed by –algorithm sm4.



Physical interpretation: Included for complete run.py coverage. It does not contribute to SM4 recovery; all operations here remain host-side software simulation.

Exact source

```
54 def aes_demo(terminal, limit, seed):
55     key, plaintext = block(aes.KEY), block(aes.PLAINTEXT)
56
57     terminal.section("AES / PDF CHAPTER 2: ENCRYPTION")
58
59     print("Key: " + key.hex().upper() + "\nPlaintext: " + plaintext.hex().upper())
60     print("AES layout: index = 4*column + row. This execution has no fault.")
61
62     clean, trace = piret_aes.trace_round8(plaintext, key, fault_round=9)
63
64     if clean != piret_aes.aes_target_encrypt(plaintext, key):
65         raise RuntimeError("AES trace differs from aes.py.")
66
67     keys = aes.key_expansion(key)
68
69     for number, round_key in enumerate(keys):
70         print(f"K{number:02d} = {aes.state_to_bytes(round_key).hex().upper()}")
71
72     initial = aes.bytes_to_state(plaintext)
73
```

```
    aes.add_round_key(initial, keys[0])

    print("Initial AddRoundKey: " + aes.state_to_bytes(initial).hex().upper())

    for number, operation, state in trace:
        if operation != "Fault point":
            print(f"R{number:02d} {operation:12s}: {state.hex().upper()}")
            terminal.pause()

    print("Correct ciphertext: " + clean.hex().upper())

    terminal.section("AES / PDF CHAPTER 3: PIRET AT ROUND-9 MIXCOLUMNS INPUT")

    print("Recover one K10 quartet per column; repeat across all four columns.")
    print("d = InvSbox[C[p] XOR k] XOR InvSbox[F[p] XOR k].")
    print("Four d values must match one of 4*255 = 1020 MixColumns difference patterns.")

    if limit >= 2:
        masks = [1, 2] + random.Random(seed).sample(list(range(3, 256)), limit - 2)
    else:
        masks = [1]

    last_key, total = bytearray(16), 0

    for column in range(4):
        survivors = None

        print(f"\nCOLUMN {column}: output/key positions {piret_aes.output_positions(column)}")

        for observation, mask in enumerate(masks, 1):
            faulty, faulty_trace = piret_aes.trace_round8(plaintext, key, 0, column, mask, fault_round=9)
            total += 1
            print(f"\nObservation {observation}: simulator XOR {mask:02X}, row 0 / column {column}.")
            print("The solver enumerates unknown fault rows/values and does not receive this mask.")
            references = {}

            for row, op, candidate_state in trace:
                references[row, op] = candidate_state

            for number, operation, state in faulty_trace:
                if number >= 9:
                    print(f"R{number:02d} {operation}")
                    terminal.compare(references[number, operation], state)
                    terminal.pause()

            terminal.pair(clean, faulty)
            attack_steps.emit(terminal, attack_steps.aes_piret(clean, faulty, column, survivors))
            # Use output differences only; the known target key is excluded from this call.
            fresh = piret_aes.key_candidates(clean, faulty, column)

            if survivors is None:
                before = None
            else:
                before = len(survivors)

            # Keep guesses satisfying every observation collected for the current column.
            if survivors is None:
                survivors = fresh
            else:
                survivors = survivors & fresh

            print(f"Candidates in this pair: {len(fresh)}; intersection: {before} -> {len(survivors)}")
            formatted_values = []

            for candidate in sorted(survivors)[:4]:
                formatted_values.append(bytes(candidate).hex().upper())

            print('First survivors: ' + ' | '.join(formatted_values))
            terminal.pause()

            if not survivors:
                raise RuntimeError("AES candidate intersection is empty.")

            if len(survivors) == 1:
                for position, value in zip(piret_aes.output_positions(column), next(iter(survivors))):
```

```
149                     last_key[position] = value
150
151                 break
152         else:
153             print(terminal.paint("AES recovery remains ambiguous. Increase --observations.", "31"))
154             return False
155
156     terminal.section("AES / PDF SECTION 3.6: REVERSE KEY EXPANSION")
157
158     print("Recovered K10: " + bytes(last_key).hex().upper())
159
160     recovered = show_aes_key_schedule(terminal, last_key)
161
162     attack_steps.emit(terminal, attack_steps.verification("aes", plaintext, recovered, clean))
163
164     if recovered != key or piret_aes.aes_target_encrypt(plaintext, recovered) != clean:
165         raise RuntimeError("AES key recovery verification failed.")
166
167     print(terminal.paint(f"AES MASTER KEY: {recovered.hex().upper()} / PASS ({total} faulty outputs)",
    ↪  "32"))
168
169     return True
```

Read each executable line

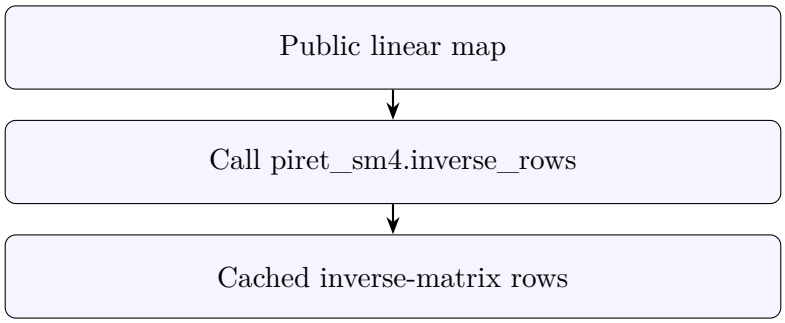
Line	Meaning and effect
54	Declare this callable and its parameters; the body runs only when called. Run the AES round-9 lesson when AES is selected; this function is not executed by –algorithm sm4.
55	Load AES simulation inputs. This line belongs to the AES branch and does not supply SM4 key material.
57	Print the named teaching-section boundary. This affects terminal/log organization only and does not advance the target or the solver.
59	Print the text/label "Plaintext: Key:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
60	Print the text/label "AES layout: index = 4*column + row. This execution has no fault.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
62	Use the AES trace adapter with fault_round=9; mask zero creates the clean reference.
64	Check that the trace adapter matches ordinary AES encryption before teaching its fault behavior.
65	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
67	Compute AES round keys for the teacher-only normal-encryption display.
69	Show each AES expanded round key in sequence.
70	Print the text/label "K =". Substitute these runtime fields: number, aes.state_to_bytes(round_key).hex().upper(). This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
72	Convert the plaintext into the AES column-major matrix for initial key addition.
74	Apply the initial AES K0 XOR in the teacher view.
76	Print the text/label "Initial AddRoundKey:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
78	Walk the clean AES transformation snapshots in execution order.
79	Exclude the adapter marker from the ordinary AES transformation display.
80	Print the text/label "R :". Substitute these runtime fields: number, operation, state.hex().upper(). This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
81	Wait for Enter only if –step is enabled. Pausing does not change a state word, create a fault or alter a candidate set.
83	Print the text/label "Correct ciphertext:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
85	Print the named teaching-section boundary. This affects terminal/log organization only and does not advance the target or the solver.

Line	Meaning and effect
87	Print the text/label "Recover one K10 quartet per column; repeat across all four columns.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
88	Print the text/label "d = InvSbox[C[p] XOR k] XOR InvSbox[F[p] XOR k].". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
89	Print the text/label "Four d values must match one of 4*255 = 1020 MixColumns difference patterns.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
91	Use the two preferred initial AES masks when the observation budget allows both.
92	Try AES masks 1 and 2 first, then reproducibly sample other distinct nonzero masks up to the limit.
93	The if-condition at line 91 was false; follow this alternative path rather than the preceding branch.
94	For a budget of one, run only the first AES mask rather than requesting extra observations.
96	Prepare sixteen AES final-key bytes and a total faulty-output counter.
98	Recover each of the four AES pre-ShiftRows key quartets in turn.
99	Mark the byte or group as unconstrained before its first observation; an empty set would instead mean inconsistency.
101	Print the text/label "COLUMN : output/key positions". Substitute these runtime fields: column, piret_aes.output_positions(column). This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
103	Request distinct faults sequentially, numbering observations from one within this target byte/group.
104	Run an AES round-9 byte-fault encryption for the selected column and mask.
105	Count this faulty AES encryption once, independently of display stages.
106	Print the text/label "Observation : simulator XOR , row 0 / column .". Substitute these runtime fields: observation, mask, column. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
107	Print the text/label "The solver enumerates unknown fault rows/values and does not receive this mask.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
108	Index clean AES snapshots by round and operation for paired fault-propagation comparisons.
110	Build the clean AES snapshot lookup explicitly from each recorded round and operation.
111	Index the clean state by its round/operation pair for later fault-propagation comparisons.
113	Visit each faulty AES transformation snapshot for the current observation.
114	Display only the AES fault round and later propagation in this part of the lesson.
115	Print the text/label "R". Substitute these runtime fields: number, operation. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
116	Compare the two supplied byte arrays visually, compute their XOR and color changed positions red. The data is not modified.
117	Wait for Enter only if –step is enabled. Pausing does not change a state word, create a fault or alter a candidate set.
119	Compare the two supplied byte arrays visually, compute their XOR and color changed positions red. The data is not modified.
120	Print exhaustive AES byte guesses and shared-error quartet patterns; this branch is not used in SM4-only execution.
122	Derive compatible AES K10 quartets from correct/faulty ciphertexts, not from the printed key.
124	Distinguish the first observation from subsequent intersections; no previous candidate set exists yet.
125	Mark the initial AES quartet history as unconstrained rather than claiming a previous finite set.
126	The if-condition at line 124 was false; follow this alternative path rather than the preceding branch.

Line	Meaning and effect
127	Record the number of hypotheses remaining before applying the new observation.
130	Distinguish the first observation from subsequent intersections; no previous candidate set exists yet.
131	Initialize this AES quartet set directly from the first observed pair.
132	The if-condition at line 130 was false; follow this alternative path rather than the preceding branch.
133	Intersect the prior hypotheses with those from the new pair; never introduce a new candidate.
135	Print the text/label "Candidates in this pair: ; intersection: ->". Substitute these runtime fields: len(fresh), before, len(survivors). This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
136	Accumulate or append the explicitly formatted hexadecimal display fragments; this does not modify cipher data.
138	Format up to four candidates for the short summary; the earlier detailed tables remain exhaustive.
139	Accumulate or append the explicitly formatted hexadecimal display fragments; this does not modify cipher data.
141	Print the text/label "First survivors: |". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
142	Wait for Enter only if –step is enabled. Pausing does not change a state word, create a fault or alter a candidate set.
144	An empty intersection means the observations and model cannot be reconciled; do not claim a key.
145	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
147	Uniqueness is the acceptance rule: one value remains compatible with all observations for this byte/group.
148	Place the unique AES tuple bytes into final ciphertext/key order using inverse ShiftRows mapping.
149	Store one uniquely recovered AES final-key byte at its true serialized position.
151	Stop this observation loop immediately after uniqueness; do not query unused masks.
152	The for-loop beginning at line 103 exhausted its observation budget without break; execute the following ambiguity result.
153	Print the text/label "AES recovery remains ambiguous. Increase –observations. 31". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
154	Tell the caller that this requested lesson did not establish verified complete recovery.
156	Print the named teaching-section boundary. This affects terminal/log organization only and does not advance the target or the solver.
158	Print the text/label "Recovered K10:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
160	Derive AES K0 from the recovered K10 through the detailed display helper.
162	Run and print recovered-key encryption/decryption consistency checks on the ordinary cipher.
164	Check the AES recovered key only after recovery: ground truth and re-encryption must both agree.
165	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
167	Print the text/label "32 AES MASTER KEY: / PASS (faulty outputs)". Substitute these runtime fields: recovered.hex().upper(), total. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
169	Return verified success to the command-line controller.

13.4 `inverse_rows`

Delegate inverse-matrix construction to the SM4 attack module.



Physical interpretation: This wrapper exists on the host for compatible call sites; it does not issue a hardware operation.

Exact source

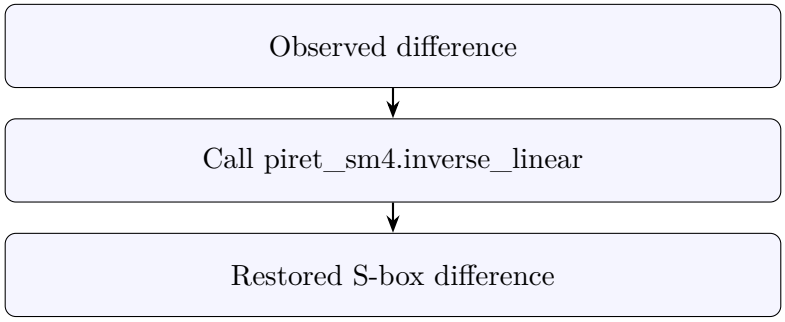
```
172 def inverse_rows():
173     """Compatibility adapter; the attack calculation lives in piret_sm4.py."""
174 
175     return piret_sm4.inverse_rows()
```

Read each executable line

Line	Meaning and effect
172	Declare this callable and its parameters; the body runs only when called. Delegate inverse-matrix construction to the SM4 attack module.
175	Delegate to the single cached implementation of inverse-matrix construction.

13.5 `inverse_linear`

Delegate inverse diffusion to the SM4 attack module.



Physical interpretation: Offline PC algebra; no new target encryption occurs.

Exact source

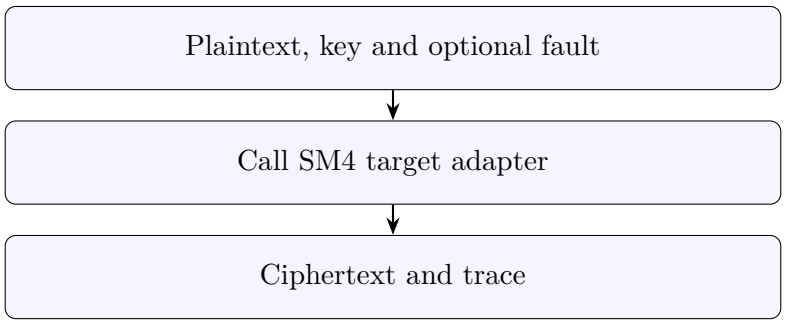
```
178 def inverse_linear(value):
179     """Compatibility adapter; the attack calculation lives in piret_sm4.py."""
180 
181     return piret_sm4.inverse_linear(value)
```

Read each executable line

Line	Meaning and effect
178	Declare this callable and its parameters; the body runs only when called. Delegate inverse diffusion to the SM4 attack module.
181	Delegate inverse diffusion; the wrapper introduces no new equation or target operation.

13.6 sm4_encrypt

Delegate a clean or faulty encryption request to the software target adapter.



Physical interpretation: This wrapper currently calls Python. A serial transport is not implemented by this function.

Exact source

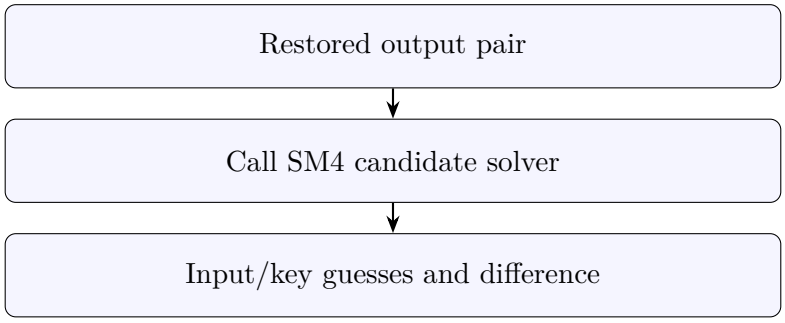
```
184 def sm4_encrypt(plaintext, key, byte=None, mask=0, fault_round=31):
185     """Terminal compatibility adapter; attack injection belongs to piret_sm4.py."""
186 
187     return piret_sm4.sm4_fault_encrypt(plaintext, key, byte, mask, fault_round)
```

Read each executable line

Line	Meaning and effect
184	Declare this callable and its parameters; the body runs only when called. Delegate a clean or faulty encryption request to the software target adapter.
187	Forward the target inputs and optional fault descriptor to the SM4 simulator, returning both output and trace.

13.7 sm4_byte_candidates

Delegate the known-mask byte equation to the attack module.



Physical interpretation: This is PC analysis of outputs, not access to hidden chip state.

Exact source

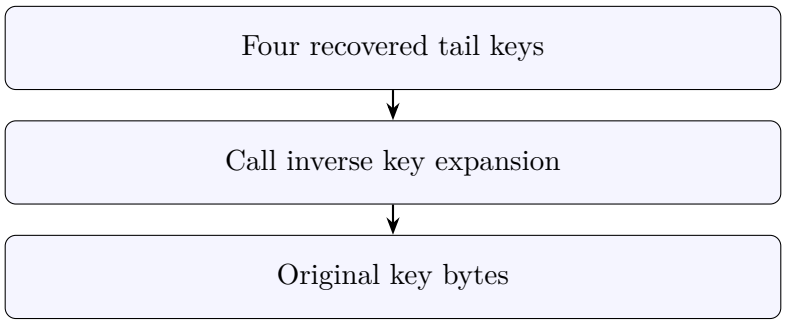
```
190 def sm4_byte_candidates(correct, faulty, byte, mask):
191     """Compatibility adapter; the attack calculation lives in piret_sm4.py."""
192 
193     return piret_sm4.sm4_byte_candidates(correct, faulty, byte, mask)
```

Read each executable line

Line	Meaning and effect
190	Declare this callable and its parameters; the body runs only when called. Delegate the known-mask byte equation to the attack module.
193	Delegate ciphertext-based candidate filtering rather than duplicate it in presentation code.

13.8 `reverse_sm4`

Delegate schedule inversion without duplicating its equations.



Physical interpretation: The host derives key material mathematically after the fault observations.

Exact source

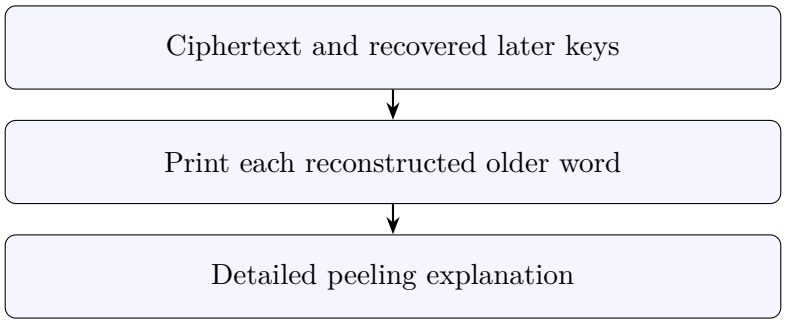
```
196 def reverse_sm4(final_keys, observer=None):
197     """Compatibility adapter; the attack calculation lives in piret_sm4.py."""
198 
199     return piret_sm4.reverse_sm4(final_keys, observer)
```

Read each executable line

Line	Meaning and effect
196	Declare this callable and its parameters; the body runs only when called. Delegate schedule inversion without duplicating its equations.
199	Delegate schedule reversal, forwarding the optional explanatory callback.

13.9 `show_sm4_peeling`

Explain every reverse round numerically for a correct or faulty output.



Physical interpretation: This reconstructs a software view on the PC. It is not a live trace of past target registers.

Exact source

```
202 def show_sm4_peeling(terminal, ciphertext, target_round, recovered_keys):
203     """Explain restored words using only subkeys already recovered by DFA."""
204 
205     view = ciphertext
206 
207     # Undo later rounds in reverse order so the target becomes the restored final round.
208     for number in range(31, target_round, -1):
209         words = struct.unpack(">4I", view)[::-1]
210 
211         round_key = recovered_keys[number]
212 
213         value = words[0] ^ words[1] ^ words[2] ^ round_key
214 
215         substituted = sm4.sbox(value)
216 
217         diffused = sm4.linear(substituted)
218 
219         # The attack engine performs the actual reversal; this helper explains it.
220         view = piret_sm4.peel_sm4(ciphertext, number-1, recovered_keys)
221 
222         previous = struct.unpack(">4I", view)[-1]
223 
224         print(f"  Undo round {number} with recovered rk{number}={round_key:08X}")
225         print(f"    A = {words[0]:08X} XOR {words[1]:08X} XOR {words[2]:08X} XOR {round_key:08X} =
        ↪  {value:08X}")
226         print(f"    tau(A) = {substituted:08X}; L(tau(A)) = {diffused:08X}")
227         print(f"    X{number} = X{number+4} XOR L(tau(A)) = {words[3]:08X} XOR {diffused:08X} =
        ↪  {previous:08X}")
228         print(f"    Restored output view: {view.hex().upper()}")
```

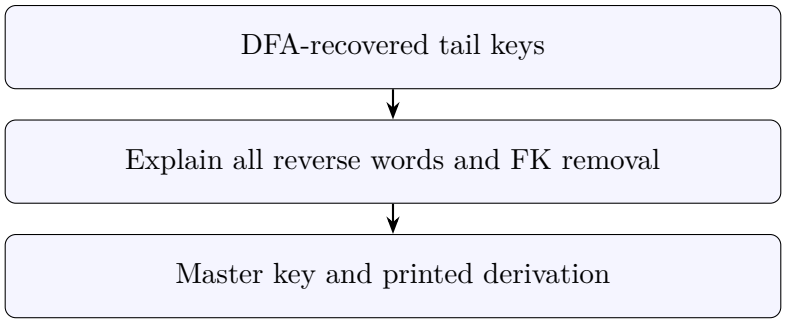
Read each executable line

Line	Meaning and effect
202	Declare this callable and its parameters; the body runs only when called. Explain every reverse round numerically for a correct or faulty output.
205	Start the explanatory peeling pass from the measured final ciphertext.
208	Undo only rounds later than the requested target round, newest first.
209	Restore chronological word order for the current reverse-round explanation.
211	Use the already recovered subkey corresponding to the later round being undone.
213	Form that round's S-box input from the three newest surviving words and its recovered key.
215	Substitute the four reconstructed input bytes using the public SM4 S-box.
217	Apply encryption diffusion L to the substituted value for the reverse-round equation.
220	Ask the actual attack helper for the restored view, so the explanatory pass cannot silently use different reversal logic.
222	Read the newly reconstructed oldest word from the last word of the ciphertext-style restored view.

Line	Meaning and effect
224	Print the text/label "Undo round with recovered rk =". Substitute these runtime fields: number, number, round_key. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
225	Print the text/label "A = XOR XOR XOR =". Substitute these runtime fields: words[0], words[1], words[2], round_key, value. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
226	Print the text/label "tau(A) = ; L(tau(A)) =". Substitute these runtime fields: substituted, diffused. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
227	Print the text/label "X = X XOR L(tau(A)) = XOR =". Substitute these runtime fields: number, number + 4, words[3], diffused, previous. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
228	Print the text/label "Restored output view:". Substitute these runtime fields: view.hex().upper(). This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.

13.10 `show_sm4_key_schedule`

Produce and print detailed SM4 inverse-schedule teaching events.



Physical interpretation: This display may repeat an already completed inverse schedule. That repetition does not collect additional faults.

Exact source

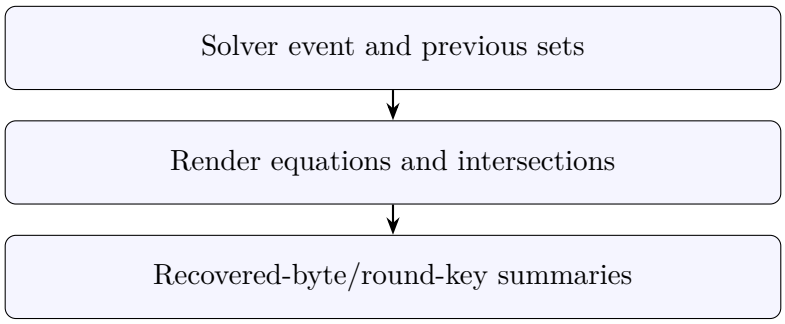
```
231 def show_sm4_key_schedule(terminal, recovered_keys):
232     """Explain inverse expansion and FK removal from the DFA-recovered tail."""
233 
234     master, events = attack_steps.sm4_schedule(recovered_keys)
235 
236     attack_steps.emit(terminal, events)
237 
238     return master
```

Read each executable line

Line	Meaning and effect
231	Declare this callable and its parameters; the body runs only when called. Produce and print detailed SM4 inverse-schedule teaching events.
234	Build all numerical inverse-schedule events and the resulting SM4 master key from recovered tail keys.
236	Print the numerical teaching events already generated by the schedule helper. This is explanatory replay, not additional fault collection.
238	Return the master-key bytes produced by the explained inverse schedule.

13.11 `show_sm4_observation`

Explain one observation, its inverse-L calculation, its candidates and its uniqueness status.



Physical interpretation: The event contains measurements and reconstructed values. Teacher traces and hidden states must not be confused with the solver inputs.

Exact source

```
def show_sm4_observation(terminal, event, recovered_bytes, start_byte=0, known_bit=None):
    """Explain one byte-DFA or Giraud event without strengthening its model."""

    number, byte = event['round'], event['byte']

    # Separate previous candidate sets by target round and byte; do not mix observations.
    identity = ('candidates', number, byte)

    attack_steps.emit(terminal, attack_steps.sm4_observation(event, recovered_bytes.get(identity),
    ↪  known_bit))

    recovered_bytes[identity] = set(event['survivors'])

    if byte == start_byte and event['observation'] == 1:
        terminal.section(f"SM4: DISCOVER rk{number} FROM NEW ROUND-{number} FAULTS")

        if number == 31:
            print("Round 31 is already last; no round needs to be undone.")
        else:
            keys = {}

            for row, candidate_value_5 in event['peeled_with'].items():
                keys[int(row)] = int(candidate_value_5, 16)

            print("Restore the target round using ONLY previously recovered later subkeys.")
            print("Correct execution:")
            show_sm4_peeling(terminal, bytes.fromhex(event['correct']), number, keys)
            print("Faulty execution (a NEW fault at the target round):")
            show_sm4_peeling(terminal, bytes.fromhex(event['faulty']), number, keys)

    print(f"\nRound {event['round']}, byte {event['byte']}, observation {event['observation']}, mask
    ↪  {event['mask']:02X}")

    terminal.pair(bytes.fromhex(event['correct']), bytes.fromhex(event['faulty']))

    print("Undo later rounds using ONLY previously recovered subkeys: " + str(event['peeled_with']))

    terminal.pair(bytes.fromhex(event['clean_view']), bytes.fromhex(event['faulty_view']))

    # Read restored output words, not hidden internal values from the target simulator.
    clean_words = struct.unpack(">4I", bytes.fromhex(event['clean_view']))

    faulty_words = struct.unpack(">4I", bytes.fromhex(event['faulty_view']))

    shift = 24 - 8 * byte

    known = ((clean_words[1] ^ clean_words[2] ^ clean_words[3]) >> shift) & 255

    delta = (event['delta_b'] >> shift) & 255
```

```
288
289     print(f"DeltaX = {clean_words[0]:08X} XOR {faulty_words[0]:08X} = {clean_words[0] ^
        ↪  faulty_words[0]:08X}")
290     print(f"DeltaB = L-inverse(DeltaX) = {event['delta_b']:08X}; selected byte difference = {delta:02X}")
291     print(f"Known byte q = byte {byte} of (X{number+1} XOR X{number+2} XOR X{number+3}) = {known:02X}")
292
293     # Unknown-bit Giraud permits eight masks; the controlled-byte DFA uses the known mask.
294     if known_bit is False:
295         allowed = (1, 2, 4, 8, 16, 32, 64, 128)
296
297         print("UNKNOWN-BIT FILTER: the simulator mask above is NOT given to the candidate solver.")
298     else:
299         allowed = (event['mask'],)
300
301         print("KNOWN-MASK FILTER: the exact error is given to the candidate solver.")
302
303     formatted_values_3 = []
304
305     for candidate_value_4 in allowed:
306         formatted_values_3.append(f'{candidate_value_4:02X}')
307
308     print('Allowed input errors: ' + ' '.join(formatted_values_3))
309     print(f"Test a=00..FF: at least one allowed e must satisfy S(a) XOR S(a XOR e) = {delta:02X}.")
310
311     for candidate in event['inputs']:
312         error = None
313
314         for allowed_error in allowed:
315             faulty_input = candidate ^ allowed_error
316             predicted_difference = sm4.SBOX[candidate] ^ sm4.SBOX[faulty_input]
317
318             if predicted_difference == delta:
319                 error = allowed_error
320                 break
321
322         if error is None:
323             raise StopIteration()
324
325         changed = candidate ^ error
326
327         print(f"  a={candidate:02X}, compatible e={error:02X}: S[{candidate:02X}] XOR S[{changed:02X}] =
        ↪  {sm4.SBOX[candidate]:02X} XOR {sm4.SBOX[changed]:02X} = {delta:02X}; rk byte = a XOR q =
        ↪  {candidate:02X} XOR {known:02X} = {candidate ^ known:02X}")
328
329     print(f"L-inverse difference: {event['delta_b']:08X}; candidates {event['before']} ->
        ↪  {event['remaining']}")
330
331     formatted_values_2 = []
332
333     for candidate_value_2 in event['survivors']:
334         formatted_values_2.append(f'{candidate_value_2:02X}')
335
336     print('Survivors: ' + ' '.join(formatted_values_2))
337
338     if event['remaining'] == 1:
339         value = event['survivors'][0]
340
341         # Store uniquely recovered bytes until the four-byte round key can be assembled.
342         recovered_bytes.setdefault(number, {})[byte] = value
343
344         print(terminal.paint(f"RECOVERED rk{number} byte {byte} = {value:02X} / UNIQUE", "32"))
345
346         if len(recovered_bytes[number]) == 4:
347             byte_values = []
348
349             for index in range(4):
350                 byte_values.append(recovered_bytes[number][index])
351
352             ordered = bytes(byte_values)
353             formatted_values = []
354
355             for candidate_value in ordered:
356                 formatted_values.append(f'{candidate_value:02X}')
357
358             print('Assemble bytes in big-endian order: ' + ' || '.join(formatted_values))
```

```
359             print(terminal.paint(f"DFA ASSEMBLED rk{number} = {ordered.hex().upper()}", "1;32"))
360
361     terminal.pause()
```

Read each executable line

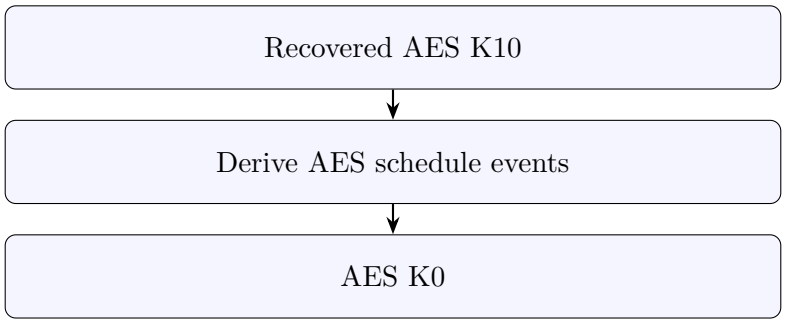
Line	Meaning and effect
241	Declare this callable and its parameters; the body runs only when called. Explain one observation, its inverse-L calculation, its candidates and its uniqueness status.
244	Read the selected target round and byte from the completed solver observation.
247	Use a compound dictionary key so candidate history for one byte cannot contaminate another round or position.
249	Print every detailed stage for this observation: restored views, inverse-L contributions, candidate tests and intersection. The previous display set is passed only for explanation.
251	Remember the current candidate set for the next detailed intersection display; this does not change the solver.
253	Print the round-level introduction only for the first observation of the first displayed byte.
254	Print the named teaching-section boundary. This affects terminal/log organization only and does not advance the target or the solver.
256	The actual last round requires no peeling; earlier target rounds must remove later rounds first.
257	Print the text/label "Round 31 is already last; no round needs to be undone.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
258	The if-condition at line 256 was false; follow this alternative path rather than the preceding branch.
259	Prepare the decoded recovered-key dictionary for the explanatory peeling pass.
261	Decode each already recovered later key carried by the event.
262	Convert string indices and hexadecimal key labels back to integers for the display calculation.
264	Print the text/label "Restore the target round using ONLY previously recovered later subkeys.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
265	Print the text/label "Correct execution:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
266	Explain correct or faulty round peeling separately; these reconstructed views remain offline host values.
267	Print the text/label "Faulty execution (a NEW fault at the target round):". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
268	Explain correct or faulty round peeling separately; these reconstructed views remain offline host values.
270	Print the text/label "Round , byte , observation , mask round byte observation mask". Substitute these runtime fields: event['round'], event['byte'], event['observation'], event['mask']. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
272	Compare the two supplied byte arrays visually, compute their XOR and color changed positions red. The data is not modified.
274	Print the text/label "Undo later rounds using ONLY previously recovered subkeys: peeled_with". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
276	Compare the two supplied byte arrays visually, compute their XOR and color changed positions red. The data is not modified.
279	Decode the clean restored output view into four big-endian words for the explanatory equations.
281	Decode the corresponding faulty restored output with exactly the same layout.
283	Map byte 0,1,2,3 to bit shifts 24,16,8,0 in a big-endian 32-bit word.
285	Recompute q from public restored words for the displayed key-byte conversion.

Line	Meaning and effect
287	Select the target byte of the already computed S-box output difference.
289	Print the text/label "DeltaX = XOR =". Substitute these runtime fields: clean_words[0], faulty_words[0], clean_words[0] ^ faulty_words[0]. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
290	Print the text/label "DeltaB = L-inverse(DeltaX) = ; selected byte difference = delta_b". Substitute these runtime fields: event['delta_b'], delta. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
291	Print the text/label "Known byte q = byte of (X XOR X XOR X) =". Substitute these runtime fields: byte, number + 1, number + 2, number + 3, known. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
294	Unknown-bit Giraud must test every one-bit mask. The default controlled-byte demonstration instead knows its mask.
295	List the eight possible single-bit errors without revealing which simulator bit was actually selected.
297	Print the text/label "UNKNOWN-BIT FILTER: the simulator mask above is NOT given to the candidate solver.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
298	The if-condition at line 294 was false; follow this alternative path rather than the preceding branch.
299	Use the exact known error for the controlled-byte or explicitly known-bit model.
301	Print the text/label "KNOWN-MASK FILTER: the exact error is given to the candidate solver.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
303	Accumulate or append the explicitly formatted hexadecimal display fragments; this does not modify cipher data.
305	Visit each value of the displayed allowed-error, survivor or recovered-byte sequence explicitly.
306	Accumulate or append the explicitly formatted hexadecimal display fragments; this does not modify cipher data.
308	Print the text/label "Allowed input errors:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
309	Print the text/label "Test a=00..FF: at least one allowed e must satisfy S(a) XOR S(a XOR e) = .". Substitute these runtime fields: delta. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
311	Explain every compatible S-box input that survived the current pair equation.
312	Start without a compatible error witness; the explicit search must find one.
314	Test each error permitted by the selected known-mask or unknown-bit model.
315	Construct the corresponding faulty S-box input for this candidate and error.
316	Calculate the predicted S-box difference for this error witness.
318	Accept the error witness only if it matches the observed byte difference.
319	Remember this compatible error before stopping the witness search.
320	Stop the permitted-error search once one compatible witness is found.
322	Report the impossible-witness condition rather than inventing an error value.
323	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
325	Reconstruct the corresponding faulty S-box input for this compatible hypothesis.
327	Print the text/label "a= , compatible e= : S[ ] XOR S[ ] = XOR = ; rk byte = a XOR q = XOR =". Substitute these runtime fields: candidate, error, candidate, changed, sm4.SBOX[candidate], sm4.SBOX[changed], delta, candidate, known, candidate ^ known. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
329	Print the text/label "L-inverse difference: ; candidates -> delta_b before remaining". Substitute these runtime fields: event['delta_b'], event['before'], event['remaining']. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
331	Accumulate or append the explicitly formatted hexadecimal display fragments; this does not modify cipher data.

Line	Meaning and effect
333	Visit each value of the displayed allowed-error, survivor or recovered-byte sequence explicitly.
334	Accumulate or append the explicitly formatted hexadecimal display fragments; this does not modify cipher data.
336	Print the text/label "Survivors:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
338	Only print a recovered-byte declaration when the accumulated intersection has exactly one value.
339	Read that unique key-byte value from the sorted event list.
342	Maintain a per-round collection of uniquely recovered bytes for final round-key assembly.
344	Print the text/label "32 RECOVERED rk byte = / UNIQUE". Substitute these runtime fields: number, byte, value. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
346	Wait until all four byte positions are known before claiming a complete 32-bit subkey.
347	Prepare an explicit byte buffer for the following comparison or recovered-key serialization.
349	Visit the four recovered byte positions in big-endian order.
350	Add the uniquely recovered byte at this position to the round-key buffer.
352	Serialize all four recovered bytes into the assembled round-key view.
353	Accumulate or append the explicitly formatted hexadecimal display fragments; this does not modify cipher data.
355	Visit each value of the displayed allowed-error, survivor or recovered-byte sequence explicitly.
356	Accumulate or append the explicitly formatted hexadecimal display fragments; this does not modify cipher data.
358	Print the text/label "Assemble bytes in big-endian order: ||". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
359	Print the text/label "1;32 DFA ASSEMBLED rk =". Substitute these runtime fields: number, ordered.hex().upper(). This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
361	Wait for Enter only if –step is enabled. Pausing does not change a state word, create a fault or alter a candidate set.

13.12 `show_aes_key_schedule`

Print the AES inverse schedule; this path is inactive in the SM4 demonstration.



Physical interpretation: Included only because it is a function in run.py. The SM4 path instead uses show_sm4_key_schedule.

Exact source

```
364 def show_aes_key_schedule(terminal, last_key):
365     """Explain every reversed AES word and assemble K9 down to K0."""
366 
367     master, events = attack_steps.aes_schedule(last_key)
368 
```

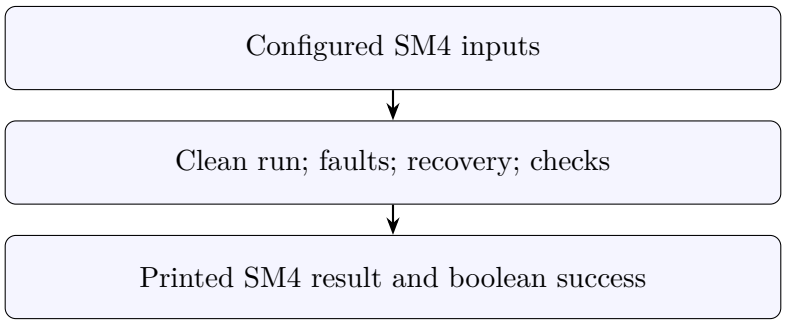
```
369     attack_steps.emit(terminal, events)
370
371     return master
```

Read each executable line

Line	Meaning and effect
364	Declare this callable and its parameters; the body runs only when called. Print the AES inverse schedule; this path is inactive in the SM4 demonstration.
367	Build AES-specific inverse-schedule teaching events; this function is inactive for SM4-only execution.
369	Print the numerical teaching events already generated by the schedule helper. This is explanatory replay, not additional fault collection.
371	Return the master-key bytes produced by the explained inverse schedule.

13.13 `sm4_demo`

Build the clean reference, define target/report callbacks, recover the key and validate the result.



Physical interpretation: This is the host experiment controller. In a board experiment the oracle would need a real transport and glitch campaign instead of the present Python target adapter.

Exact source

```
374 def sm4_demo(terminal, limit, start_byte=0):
375     """Recover all four final subkeys through faults, then the complete key."""
376
377     key, plaintext = block(sm4.KEY), block(sm4.PLAINTEXT)
378
379     correct, trace = sm4_encrypt(plaintext, key)
380
381     terminal.section("SM4: NORMAL ENCRYPTION")
382
383     for item in trace:
384         print(f"Round {item['round']:02d}: A={item['a']:08X} B={item['b']:08X} L(B)={item['linear']:08X}
        ↪  new={item['new']:08X}")
385
386         terminal.pause()
387
388     print("Ciphertext: " + correct.hex().upper())
389
390     terminal.section("SM4: FULL MASTER-KEY DFA / ROUNDS 31 -> 28")
391
392     print("Controlled faults in EACH final round are required. No reference subkeys are supplied.")
393     print("For each byte: undo L, test S(a) XOR S(a XOR mask), then convert a to a subkey guess.")
394     print("Repeat independent faults and intersect guesses. Only one survivor identifies a byte.")
395
396     recovered_bytes = {}
397
398     # Run the real SM4 cipher with an external fault callback and show teacher-only propagation.
```

```
399     def oracle(number, byte, mask):
400         faulty, fault_trace = piret_sm4.sm4_fault_encrypt(plaintext, key, byte, mask, number)
401 
402         attack_steps.emit(terminal, attack_steps.sm4_fault_trace(trace, fault_trace, number, byte, mask))
403 
404         return faulty
405 
406     def report(event):
407         show_sm4_observation(terminal, event, recovered_bytes, start_byte)
408 
409     result = piret_sm4.recover_sm4_master_key(correct, oracle, range(1, limit+1), start_byte,
        ↪  observer=report)
410 
411     if not result['complete']:
412         print("SM4 recovery remains ambiguous. Increase --observations.")
413 
414         return False
415 
416     for number, value in result['round_keys'].items():
417         print(terminal.paint(f"DFA RECOVERED rk{number} = {value:08X} / PASS", "32"))
418 
419     recovered = show_sm4_key_schedule(terminal, result['round_keys'])
420 
421     if recovered != result['master_key']:
422         raise RuntimeError("Displayed inverse schedule disagrees with the recovery engine.")
423 
424     attack_steps.emit(terminal, attack_steps.verification("sm4", plaintext, recovered, correct))
425 
426     verified_ciphertext = sm4.sm4_encrypt_block(plaintext, recovered)
427 
428     if recovered != key or verified_ciphertext != correct:
429         raise RuntimeError("SM4 full master-key verification failed.")
430 
431     terminal.section("SM4: VERIFY THE RECOVERED MASTER KEY")
432 
433     print("Plaintext:                " + plaintext.hex().upper())
434     print("Expected ciphertext:      " + correct.hex().upper())
435     print("Recovered-key ciphertext: " + verified_ciphertext.hex().upper())
436     print(terminal.paint("SM4 MASTER KEY: " + recovered.hex().upper() + " / PASS", "32"))
437     print(f"{result['used']} faulty outputs. All four subkeys came from DFA, not references.")
438 
439     return True
```

Read each executable line

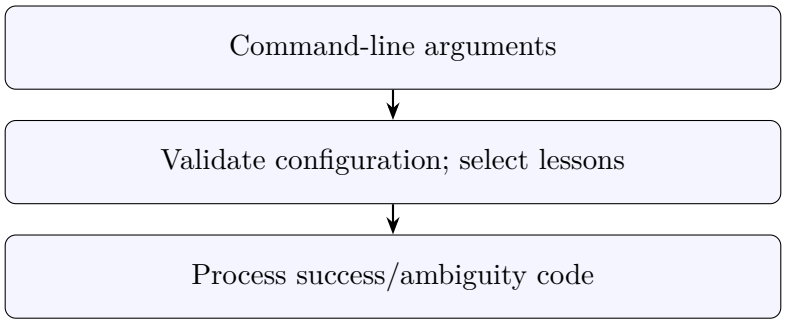
Line	Meaning and effect
374	Declare this callable and its parameters; the body runs only when called. Build the clean reference, define target/report callbacks, recover the key and validate the result.
377	Read and validate the configured target key and plaintext for simulation. The solver does not receive the key.
379	Run a clean reference encryption with no fault selected and retain teacher-only round records.
381	Print the named teaching-section boundary. This affects terminal/log organization only and does not advance the target or the solver.
383	Print each of the thirty-two clean encryption-round records in order.
384	Print the text/label "Round : A= B= L(B)= new= round a b linear new". Substitute these runtime fields: item['round'], item['a'], item['b'], item['linear'], item['new']. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
386	Wait for Enter only if –step is enabled. Pausing does not change a state word, create a fault or alter a candidate set.
388	Print the text/label "Ciphertext:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
390	Print the named teaching-section boundary. This affects terminal/log organization only and does not advance the target or the solver.

Line	Meaning and effect
392	Print the text/label "Controlled faults in EACH final round are required. No reference subkeys are supplied.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
393	Print the text/label "For each byte: undo L, test S(a) XOR S(a XOR mask), then convert a to a subkey guess.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
394	Print the text/label "Repeat independent faults and intersect guesses. Only one survivor identifies a byte.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
396	Start presentation-only histories for candidate sets and uniquely recovered round-key bytes.
399	Define a target-query callback closing over the plaintext and simulated secret key. Its public result is only a ciphertext.
400	Perform one new target encryption with the selected fault. The detailed trace is for teaching, not solving.
402	Print teacher-only fault propagation from the target trace. This exposes intermediate values for learning, but none are returned as solver inputs.
404	Give the attacker only the target ciphertext; do not return its trace or target key.
406	Define the presentation callback invoked after each solver observation; it does not generate faults or choose candidates.
407	Render a solver event without taking control of its candidate computation.
409	Run full output-based SM4 recovery with mask budget 1..limit, selected starting byte and a reporting callback.
411	Detect an observation budget exhausted before full uniqueness; do not continue to a claimed master key.
412	Print the text/label "SM4 recovery remains ambiguous. Increase –observations.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
414	Tell the caller that this requested lesson did not establish verified complete recovery.
416	Summarize only the four subkeys actually recovered through fault observations.
417	Print the text/label "32 DFA RECOVERED rk = / PASS". Substitute these runtime fields: number, value. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
419	Repeat inverse expansion through the detailed teaching view and retain its reconstructed master key.
421	Cross-check the detailed display's inverse schedule against the recovery engine's result.
422	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
424	Run and print recovered-key encryption/decryption consistency checks on the ordinary cipher.
426	Re-encrypt the known plaintext with the recovered key using the ordinary SM4 cipher.
428	Validate both simulation ground-truth equality and the original correct ciphertext, only after recovery.
429	Stop this path with the stated exception. The message identifies the violated input, model or verification condition; no recovered key is accepted by this line.
431	Print the named teaching-section boundary. This affects terminal/log organization only and does not advance the target or the solver.
433	Print the text/label "Plaintext:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
434	Print the text/label "Expected ciphertext:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
435	Print the text/label "Recovered-key ciphertext:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
436	Print the text/label "32 / PASS SM4 MASTER KEY:". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
437	Print the text/label "faulty outputs. All four subkeys came from DFA, not references. used". Substitute these runtime fields: result['used']. This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.

Line	Meaning and effect
439	Return verified success to the command-line controller.

13.14 `main`

Choose AES, SM4 or both and aggregate their exit status.



Physical interpretation: All selection, printing and log handling runs on the PC. –algorithm sm4 skips the AES lesson, although configuration validation checks both modules.

Exact source

```
442 def main(argv=None):
443     parser = argparse.ArgumentParser(description=__doc__)
444 
445     parser.add_argument("--algorithm", choices=("all", "aes", "sm4"), default="all")
446     parser.add_argument("--step", action="store_true", help="Pause after each state/observation.")
447     parser.add_argument("--color", choices=("auto", "always", "never"), default="always")
448     parser.add_argument("--observations", type=int, default=8, help="Max observations per AES column / SM4
        ↪  byte (1..255).")
449     parser.add_argument("--seed", type=int, default=42)
450 
451     args = parser.parse_args(argv)
452 
453     if not 1 <= args.observations <= 255:
454         parser.error("--observations must be 1..255.")
455 
456     try:
457         for value in (aes.KEY, aes.PLAINTEXT, sm4.KEY, sm4.PLAINTEXT):
458             block(value)
459     except ValueError as error:
460         parser.error(str(error))
461 
462     terminal = LessonTerminal(args.color, args.step)
463 
464     print("AES_SM4_DFA.pdf / chapters 2-6 / software demonstration")
465     print("RED marks bytes changed by the injected fault, including propagated changes.")
466 
467     # Track success independently for the AES and SM4 demonstrations requested by the command.
468     statuses = []
469 
470     if args.algorithm in ("all", "aes"):
471         statuses.append(aes_demo(terminal, args.observations, args.seed))
472 
473     if args.algorithm in ("all", "sm4"):
474         statuses.append(sm4_demo(terminal, args.observations))
475 
476     if all(statuses):
477         result_value = 0
478     else:
479         result_value = 2
480 
481     return result_value
```

Read each executable line

Line	Meaning and effect
442	Declare this callable and its parameters; the body runs only when called. Choose AES, SM4 or both and aggregate their exit status.
443	Create the command-line option parser using the module description for help text.
445	Register –algorithm. Select the allowed cipher branch for this entry point.
446	Register –step. Enable pauses at teaching boundaries.
447	Register –color. Select automatic, forced or disabled ANSI colors.
448	Register –observations. Set the per-byte or per-column observation budget.
449	Register –seed. Used for AES mask sampling in run.py; not used for SM4 mask order.
451	Decode the supplied argument list (or process command line) and enforce the declared argument types and choices.
453	Reject an observation budget outside 1..255 before requesting any faulty outputs.
454	Report invalid configuration and terminate argument processing rather than starting an invalid experiment.
456	Begin a protected configuration-validation block; the associated except path turns invalid input into a parser error.
457	Validate both cipher configurations even if just one lesson was requested; this is configuration checking, not AES execution.
458	Require this configured key or plaintext to decode as a complete block.
459	Catch the named validation exception and report its message through the command-line parser.
460	Report invalid configuration and terminate argument processing rather than starting an invalid experiment.
462	Configure presentation colors and optional pauses; these choices do not affect the cryptographic equations.
464	Print the text/label "AES_SM4_DFA.pdf / chapters 2-6 / software demonstration". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
465	Print the text/label "RED marks bytes changed by the injected fault, including propagated changes.". This is presentation output, also mirrored to the active log; it performs no target query and changes no candidates.
468	Collect independent success flags for the requested algorithm demonstrations.
470	Execute the AES branch only when requested. It is skipped by –algorithm sm4.
471	Run the AES lesson and store its success flag; not executed in the SM4-only command.
473	Execute the SM4 demonstration for either an explicit SM4 selection or the combined default.
474	Run the SM4 lesson with its observation budget and store whether full recovery verified.
476	Check the already collected success flags for all requested algorithm demonstrations.
477	Select process success because all requested demonstrations verified.
478	The if-condition at line 476 was false; follow this alternative path rather than the preceding branch.
479	Select process ambiguity status because a requested demonstration did not fully recover its key.
481	Return the word assembled from all thirty-two inverse-row parity calculations.

Appendix A

All 32 clean encryption rounds

Generated from `sm4_fault_encrypt()` with no fault selected. Every row corresponds to the normal terminal trace for this vector.

Round	A	$\tau(A)$	$L(\tau(A))$	New word
0	F002C39E	18E992B1	26D99622	27FAD345
1	EE1470AC	C62AD40D	2820815D	A18B4CB2
2	AA4F1C7D	23A849C4	EF1D58B2	11C1E22A
3	EC195DAA	C5445623	BA47D0FE	CC13E2EE
4	4A2A2C82	3C0BED8A	DF868890	F87C5BD5
5	52C45770	81BB8BD4	92A94BE5	33220757
6	B1F637DF	DB4D95B0	663520BD	77F4C297
7	98DCAF44	ADB86FF3	B6851005	7A96F2EB
8	9B600757	301EB78B	DFA69BAA	27DAC07F
9	9DE0BDBE	8C896C5B	71FF084E	42DD0F19
10	DC966E60	B81A781E	CF511895	B8A5DA02
11	A3474E33	2EBA4FA9	EAE7D511	907127FA
12	0381926D	FEBF5D21	AC4FEBFC	8B952B83
13	939943CC	A493FC1F	96F67340	D42B7C59
14	8B75648F	CED363A1	97598233	2FFC5831
15	60069A4A	1E3D323C	66EF4F72	F69E6888
16	DC69F8C8	B8227911	24B11947	AF2432C4
17	05F35DDE	E1EC56B4	3935B407	ED1EC85E
18	7823DB74	4CF4129F	7A5FE213	55A3BA22
19	85BD0481	C76CCCBF	E4D57022	124B18AA
20	42680EC9	A7252CD9	C5C3409B	6AE7725F
21	B5C5D18D	FDDDC161	19D569A7	F4CBA1F9
22	4B725B6C	19464B01	486E4032	1DCDFA10
23	1A00D498	13D6A5AD	3DBD1EA9	2FF60603
24	716B85E6	003BC777	85153D83	EFF24FDC
25	C0E8A67F	8D65CA9E	9B2FCA8C	6FE46B75
26	A1C2A841	F6AFC007	94F9AABD	893450AD
27	F85A7885	790F4CC7	5465894F	7B938F4C
28	DFCE82C0	B05A8A8D	BC9C0D9A	536E4246
29	C3E0A931	928929B3	E957823A	86B3E94F
30	AF8156A0	6FBFDA1D	5B32C6F3	D206965E
31	96FF9D45	1A488C73	138D5078	681EDF34

Appendix B

All observations used in full recovery

The fresh set uses one ciphertext pair. The intersection uses every preceding observation for the same round and byte. The displayed bytes are hexadecimal. q is computed after peeling when necessary.

Round	Byte	e	d	q	Fresh key guesses	Intersection
31	0	01	4F	07	90 91	90 91
31	0	02	81	07	70 72 91 93	91
31	1	01	71	DB	24 25	24 25
31	1	02	83	DB	24 26	24
31	2	01	79	3D	A0 A1	A0 A1
31	2	02	6F	3D	A0 A2	A0
31	3	01	80	57	12 13	12 13
31	3	02	C9	57	10 12	12
30	0	01	21	AE	00 01	00 01
30	0	02	3C	AE	01 03	01
30	1	01	55	4E	CE CF	CE CF
30	1	02	6D	4E	CD CF	CF
30	2	01	51	24	72 73	72 73
30	2	02	AB	24	70 72	72
30	3	01	EB	45	E4 E5	E4 E5
30	3	02	FF	45	E5 E7	E5
29	0	01	3D	A1	62 63	62 63
29	0	02	89	A1	60 62	62
29	1	01	E0	C9	28 29	28 29
29	1	02	1E	C9	29 2B	29
29	2	01	E9	9D	34 35	34 35
29	2	02	82	9D	34 36	34
29	3	01	57	A7	96 97	96 97
29	3	02	1A	A7	94 96	96
28	0	01	04	9D	42 43	42 43
28	0	02	55	9D	40 42	42
28	1	01	82	43	8C 8D	8C 8D
28	1	02	45	43	8D 8F	8D
28	2	01	58	B4	36 37	36 37
28	2	02	60	B4	34 36	36
28	3	01	96	94	54 55	54 55
28	3	02	22	94	54 56	54

Appendix C

All inverse-L bit contributions for the first fault

Input $\Delta X =$ `734E3D3D`. Bit 0 is the least significant bit. The final running XOR is $\Delta B =$ `4F000000`.

Bit	Input	Inverse basis	Contribution	Running XOR
0	1	41455115	41455115	41455115
1	0	828AA22A	00000000	41455115
2	1	05154455	05154455	44501540
3	1	0A2A88AA	0A2A88AA	4E7A9DEA
4	1	14551154	14551154	5A2F8CBE
5	1	28AA22A8	28AA22A8	7285AE16
6	0	51544550	00000000	7285AE16
7	0	A2A88AA0	00000000	7285AE16
8	1	45511541	45511541	37D4BB57
9	0	8AA22A82	00000000	37D4BB57
10	1	15445505	15445505	2290EE52
11	1	2A88AA0A	2A88AA0A	08184458
12	1	55115414	55115414	5D09104C
13	1	AA22A828	AA22A828	F72BB864
14	0	54455051	00000000	F72BB864
15	0	A88AA0A2	00000000	F72BB864
16	0	51154145	00000000	F72BB864
17	1	A22A828A	A22A828A	55013AEE
18	1	44550515	44550515	11543FFB
19	1	88AA0A2A	88AA0A2A	99FE35D1
20	0	11541455	00000000	99FE35D1
21	0	22A828AA	00000000	99FE35D1
22	1	45505154	45505154	DCAE6485
23	0	8AA0A2A8	00000000	DCAE6485
24	1	15414551	15414551	C9EF21D4
25	1	2A828AA2	2A828AA2	E36DAB76
26	0	55051544	00000000	E36DAB76
27	0	AA0A2A88	00000000	E36DAB76
28	1	54145511	54145511	B779FE67
29	1	A828AA22	A828AA22	1F515445
30	1	50515445	50515445	4F000000
31	0	A0A2A88A	00000000	4F000000

Appendix D

All 256 candidate inputs for the first two faults

Both observations concern rk31 byte 0. The known byte is $q = 07$. Column $a \oplus 07$ converts an input guess to a key guess. Only inputs marked ACCEPT contribute to that observation's fresh set. The solver intersects the resulting *key* sets.

a	$S(a)$	d for $e = 01$	$d = 4F$?	d for $e = 02$	$d = 81$?	$a \oplus 07$
00	D6	46	reject	3F	reject	07
01	90	46	reject	6E	reject	06
02	E9	17	reject	3F	reject	05
03	FE	17	reject	6E	reject	04
04	CC	2D	reject	F1	reject	03
05	E1	2D	reject	56	reject	02
06	3D	8A	reject	F1	reject	01
07	B7	8A	reject	56	reject	00
08	16	A0	reject	02	reject	0F
09	B6	A0	reject	74	reject	0E
0A	14	D6	reject	02	reject	0D
0B	C2	D6	reject	74	reject	0C
0C	28	D3	reject	04	reject	0B
0D	FB	D3	reject	FE	reject	0A
0E	2C	29	reject	04	reject	09
0F	05	29	reject	FE	reject	08
10	2B	4C	reject	B1	reject	17
11	67	4C	reject	11	reject	16
12	9A	EC	reject	B1	reject	15
13	76	EC	reject	11	reject	14
14	2A	94	reject	2E	reject	13
15	BE	94	reject	7D	reject	12
16	04	C7	reject	2E	reject	11
17	C3	C7	reject	7D	reject	10
18	AA	EE	reject	B9	reject	1F
19	44	EE	reject	62	reject	1E
1A	13	35	reject	B9	reject	1D
1B	26	35	reject	62	reject	1C
1C	49	CF	reject	4F	reject	1B
1D	86	CF	reject	1F	reject	1A
1E	06	9F	reject	4F	reject	19
1F	99	9F	reject	1F	reject	18
20	9C	DE	reject	CC	reject	27
21	42	DE	reject	B6	reject	26
22	50	A4	reject	CC	reject	25
23	F4	A4	reject	B6	reject	24
24	91	7E	reject	09	reject	23

a	$S(a)$	d for $e = 01$	$d = 4F$?	d for $e = 02$	$d = 81$?	$a \oplus 07$
25	EF	7E	reject	95	reject	22
26	98	E2	reject	09	reject	21
27	7A	E2	reject	95	reject	20
28	33	67	reject	38	reject	2F
29	54	67	reject	17	reject	2E
2A	0B	48	reject	38	reject	2D
2B	43	48	reject	17	reject	2C
2C	ED	22	reject	41	reject	2B
2D	CF	22	reject	AD	reject	2A
2E	AC	CE	reject	41	reject	29
2F	62	CE	reject	AD	reject	28
30	E4	57	reject	F8	reject	37
31	B3	57	reject	1A	reject	36
32	1C	B5	reject	F8	reject	35
33	A9	B5	reject	1A	reject	34
34	C9	C1	reject	21	reject	33
35	08	C1	reject	9D	reject	32
36	E8	7D	reject	21	reject	31
37	95	7D	reject	9D	reject	30
38	80	5F	reject	14	reject	3F
39	DF	5F	reject	25	reject	3E
3A	94	6E	reject	14	reject	3D
3B	FA	6E	reject	25	reject	3C
3C	75	FA	reject	4A	reject	3B
3D	8F	FA	reject	29	reject	3A
3E	3F	99	reject	4A	reject	39
3F	A6	99	reject	29	reject	38
40	47	40	reject	E0	reject	47
41	07	40	reject	FB	reject	46
42	A7	5B	reject	E0	reject	45
43	FC	5B	reject	FB	reject	44
44	F3	80	reject	E4	reject	43
45	73	80	reject	C9	reject	42
46	17	AD	reject	E4	reject	41
47	BA	AD	reject	C9	reject	40
48	83	DA	reject	BF	reject	4F
49	59	DA	reject	40	reject	4E
4A	3C	25	reject	BF	reject	4D
4B	19	25	reject	40	reject	4C
4C	E6	63	reject	A9	reject	4B
4D	85	63	reject	2D	reject	4A
4E	4F	E7	reject	A9	reject	49
4F	A8	E7	reject	2D	reject	48
50	68	03	reject	E9	reject	57
51	6B	03	reject	D9	reject	56
52	81	33	reject	E9	reject	55
53	B2	33	reject	D9	reject	54
54	71	15	reject	AB	reject	53
55	64	15	reject	EF	reject	52
56	DA	51	reject	AB	reject	51
57	8B	51	reject	EF	reject	50
58	F8	13	reject	F7	reject	5F
59	EB	13	reject	A0	reject	5E
5A	0F	44	reject	F7	reject	5D
5B	4B	44	reject	A0	reject	5C
5C	70	26	reject	ED	reject	5B
5D	56	26	reject	63	reject	5A
5E	9D	A8	reject	ED	reject	59

a	$S(a)$	d for $e = 01$	$d = 4F$?	d for $e = 02$	$d = 81$?	$a \oplus 07$
5F	35	A8	reject	63	reject	58
60	1E	3A	reject	10	reject	67
61	24	3A	reject	7A	reject	66
62	0E	50	reject	10	reject	65
63	5E	50	reject	7A	reject	64
64	63	3B	reject	B2	reject	63
65	58	3B	reject	FA	reject	62
66	D1	73	reject	B2	reject	61
67	A2	73	reject	FA	reject	60
68	25	07	reject	59	reject	6F
69	22	07	reject	19	reject	6E
6A	7C	47	reject	59	reject	6D
6B	3B	47	reject	19	reject	6C
6C	01	20	reject	79	reject	6B
6D	21	20	reject	A6	reject	6A
6E	78	FF	reject	79	reject	69
6F	87	FF	reject	A6	reject	68
70	D4	D4	reject	92	reject	77
71	00	D4	reject	57	reject	76
72	46	11	reject	92	reject	75
73	57	11	reject	57	reject	74
74	9F	4C	reject	B8	reject	73
75	D3	4C	reject	81	ACCEPT	72
76	27	75	reject	B8	reject	71
77	52	75	reject	81	ACCEPT	70
78	4C	7A	reject	4E	reject	7F
79	36	7A	reject	D1	reject	7E
7A	02	E5	reject	4E	reject	7D
7B	E7	E5	reject	D1	reject	7C
7C	A0	64	reject	68	reject	7B
7D	C4	64	reject	5A	reject	7A
7E	C8	56	reject	68	reject	79
7F	9E	56	reject	5A	reject	78
80	EA	55	reject	60	reject	87
81	BF	55	reject	6D	reject	86
82	8A	58	reject	60	reject	85
83	D2	58	reject	6D	reject	84
84	40	87	reject	78	reject	83
85	C7	87	reject	72	reject	82
86	38	8D	reject	78	reject	81
87	B5	8D	reject	72	reject	80
88	A3	54	reject	51	reject	8F
89	F7	54	reject	39	reject	8E
8A	F2	3C	reject	51	reject	8D
8B	CE	3C	reject	39	reject	8C
8C	F9	98	reject	EC	reject	8B
8D	61	98	reject	C0	reject	8A
8E	15	B4	reject	EC	reject	89
8F	A1	B4	reject	C0	reject	88
90	E0	4E	reject	BD	reject	97
91	AE	4E	reject	0A	reject	96
92	5D	F9	reject	BD	reject	95
93	A4	F9	reject	0A	reject	94
94	9B	AF	reject	81	ACCEPT	93
95	34	AF	reject	61	reject	92
96	1A	4F	ACCEPT	81	ACCEPT	91
97	55	4F	ACCEPT	61	reject	90
98	AD	3E	reject	9F	reject	9F

a	$S(a)$	d for $e = 01$	$d = 4F$?	d for $e = 02$	$d = 81$?	$a \oplus 07$
99	93	3E	reject	A3	reject	9E
9A	32	02	reject	9F	reject	9D
9B	30	02	reject	A3	reject	9C
9C	F5	79	reject	44	reject	9B
9D	8C	79	reject	6F	reject	9A
9E	B1	52	reject	44	reject	99
9F	E3	52	reject	6F	reject	98
A0	1D	EB	reject	FF	reject	A7
A1	F6	EB	reject	D8	reject	A6
A2	E2	CC	reject	FF	reject	A5
A3	2E	CC	reject	D8	reject	A4
A4	82	E4	reject	48	reject	A3
A5	66	E4	reject	06	reject	A2
A6	CA	AA	reject	48	reject	A1
A7	60	AA	reject	06	reject	A0
A8	C0	E9	reject	E3	reject	AF
A9	29	E9	reject	82	reject	AE
AA	23	88	reject	E3	reject	AD
AB	AB	88	reject	82	reject	AC
AC	0D	5E	reject	43	reject	AB
AD	53	5E	reject	3C	reject	AA
AE	4E	21	reject	43	reject	A9
AF	6F	21	reject	3C	reject	A8
B0	D5	0E	reject	E2	reject	B7
B1	DB	0E	reject	9E	reject	B6
B2	37	72	reject	E2	reject	B5
B3	45	72	reject	9E	reject	B4
B4	DE	23	reject	50	reject	B3
B5	FD	23	reject	D2	reject	B2
B6	8E	A1	reject	50	reject	B1
B7	2F	A1	reject	D2	reject	B0
B8	03	FC	reject	69	reject	BF
B9	FF	FC	reject	8D	reject	BE
BA	6A	18	reject	69	reject	BD
BB	72	18	reject	8D	reject	BC
BC	6D	01	reject	36	reject	BB
BD	6C	01	reject	3D	reject	BA
BE	5B	0A	reject	36	reject	B9
BF	51	0A	reject	3D	reject	B8
C0	8D	96	reject	22	reject	C7
C1	1B	96	reject	89	reject	C6
C2	AF	3D	reject	22	reject	C5
C3	92	3D	reject	89	reject	C4
C4	BB	66	reject	07	reject	C3
C5	DD	66	reject	A2	reject	C2
C6	BC	C3	reject	07	reject	C1
C7	7F	C3	reject	A2	reject	C0
C8	11	C8	reject	4D	reject	CF
C9	D9	C8	reject	98	reject	CE
CA	5C	1D	reject	4D	reject	CD
CB	41	1D	reject	98	reject	CC
CC	1F	0F	reject	45	reject	CB
CD	10	0F	reject	C8	reject	CA
CE	5A	82	reject	45	reject	C9
CF	D8	82	reject	C8	reject	C8
D0	0A	CB	reject	3B	reject	D7
D1	C1	CB	reject	49	reject	D6
D2	31	B9	reject	3B	reject	D5

a	$S(a)$	d for $e = 01$	$d = 4F$?	d for $e = 02$	$d = 81$?	$a \oplus 07$
D3	88	B9	reject	49	reject	D4
D4	A5	68	reject	DE	reject	D3
D5	CD	68	reject	70	reject	D2
D6	7B	C6	reject	DE	reject	D1
D7	BD	C6	reject	70	reject	D0
D8	2D	59	reject	FD	reject	DF
D9	74	59	reject	66	reject	DE
DA	D0	C2	reject	FD	reject	DD
DB	12	C2	reject	66	reject	DC
DC	B8	5D	reject	0C	reject	DB
DD	E5	5D	reject	55	reject	DA
DE	B4	04	reject	0C	reject	D9
DF	B0	04	reject	55	reject	D8
E0	89	E0	reject	1E	reject	E7
E1	69	E0	reject	23	reject	E6
E2	97	DD	reject	1E	reject	E5
E3	4A	DD	reject	23	reject	E4
E4	0C	9A	reject	7B	reject	E3
E5	96	9A	reject	E8	reject	E2
E6	77	09	reject	7B	reject	E1
E7	7E	09	reject	E8	reject	E0
E8	65	DC	reject	94	reject	EF
E9	B9	DC	reject	B0	reject	EE
EA	F1	F8	reject	94	reject	ED
EB	09	F8	reject	B0	reject	EC
EC	C5	AB	reject	03	reject	EB
ED	6E	AB	reject	EA	reject	EA
EE	C6	42	reject	03	reject	E9
EF	84	42	reject	EA	reject	E8
F0	18	E8	reject	65	reject	F7
F1	F0	E8	reject	1C	reject	F6
F2	7D	91	reject	65	reject	F5
F3	EC	91	reject	1C	reject	F4
F4	3A	E6	reject	77	reject	F3
F5	DC	E6	reject	FC	reject	F2
F6	4D	6D	reject	77	reject	F1
F7	20	6D	reject	FC	reject	F0
F8	79	97	reject	26	reject	FF
F9	EE	97	reject	D0	reject	FE
FA	5F	61	reject	26	reject	FD
FB	3E	61	reject	D0	reject	FC
FC	D7	1C	reject	EE	reject	FB
FD	CB	1C	reject	83	reject	FA
FE	39	71	reject	EE	reject	F9
FF	48	71	reject	83	reject	F8

Appendix E

All 32 inverse key-schedule calculations

$U = K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i$. The final column is $K_i = K_{i+4} \oplus L'(\tau(U))$. Computation runs from $i = 31$ down to zero.

i	CK_i	U	$\tau(U)$	$L'(\tau(U))$	K_i
31	646B7279	4500025E	73D6E99D	605CAC93	F1780C81
30	484F565D	9993581E	93A4F806	0FEDF80E	0E228AEB
29	2C333A41	91E48A7F	AE0CF29E	7F082126	1D2115B0
28	10171E25	F26C8DFF	7D016148	F516EE58	B79BD80C
27	F4FB0209	5063455E	685E739D	6899F1AF	99E1FD2E
26	D8DFE6ED	EB84D67F	09407B9E	C9371A8B	C7159060
25	BCC3CAD1	55AC7F93	640D9EA4	85EB14EA	98CA015A
24	A0A7AEB5	6699C2A1	D193AFF6	5F05BC13	E89E641F
23	848B9299	33CA67BC	A95CA26D	0BC5B917	92244439
22	686F767D	8A1F5701	F2998B90	0B92D906	CC874966
21	4C535A61	FA6E3321	5F78A942	EB7F5EF9	73B55FA3
20	30373E45	1D216CB9	864201FF	39BED037	D120B428
19	141B2229	7A0980C4	02B6EABB	8260D198	104495A1
18	F8FF060D	4A2E7827	3CAC4C7A	883D5DC9	44BA14AF
17	DCE3EAF1	593DDFD7	EB8FB0BD	436DCA14	30D895B7
16	C0C7CED5	A4E1DA6C	8269D001	B8A8D4A4	6988608C
15	A4ABB2B9	B941532D	FF07B2CF	6EA1CEF6	7EE55B57
14	888F969D	AF3A38F1	6F9480F0	87BD4742	C30753ED
13	6C737A81	B81912B7	03449A2F	8780D80A	B7584DBD
12	50575E65	5AED1B62	0F6E260E	CCA850F0	A520307C
11	343B4249	E5446C65	96F30158	5A936A06	24763151
10	181F262D	2E116ABD	AC677C6C	75BCDA5E	B6BB89B3
9	FC030A11	CBEE828F	41C68AA1	C03241DC	776A0C61
8	E0E7EEF5	05405A76	E1470F27	93535088	367360F4
7	C4CBD2D9	336937FF	A9229548	5FDF1126	7BA92077
6	A8AFB6BD	921FFA5F	5D995F35	ECD13829	5A6AB19A
5	8C939AA1	9B236BB8	30F43B03	360C2700	41662B61
4	70777E85	10D2C409	2B31BBB6	C752E60D	F12186F9
3	545B6269	BE767E6B	5B27C83B	BF8D30BB	C42410CC
2	383F464D	4C5CFB19	E6703E44	C3CB9A95	99A12B0F
1	1C232A31	B087970B	D5B555C2	9E67D5DE	DF01FEBF
0	00070E15	8283CB69	8AD24122	53B37958	A292FFA1

Appendix F

Supporting source excerpts with real line numbers

These supporting excerpts are regenerated from the actual files whenever this independent PDF is built. The preceding line-by-line chapters already cover all functions in `piret_sm4.py` and `run.py`. This appendix provides the cipher hooks, detailed-event helpers and logging code they use, plus the shared terminal base class. The imports of `aes` in the SM4 attack provide block-length validation only; no AES key or AES attack equation participates in SM4 recovery.

`generated/example.json` records the public vector, all observations, recovered keys, clean trace and SHA-256 hashes of the relevant source files. Measured output tables and quoted line numbers can therefore be checked against the code used for this build.

F.1 Source excerpts: `piret_aes.py`

F.1.1 `Terminal`

Source lines 444–534.

```
444 class Terminal:
445     """ANSI styling is optional; redirected output stays readable by default."""
446     def __init__(self, color="auto", step=False):
447         # Respect explicit color settings, terminal capability and the NO_COLOR environment flag.
448         self.color = color == "always" or color == "auto" and sys.stdout.isatty() and "NO_COLOR" not in
            ↪  os.environ
449 
450         self.step = step
451 
452         if self.color and os.name == "nt":
453             # Enable virtual-terminal processing without requiring colorama.
454             import ctypes
455             kernel = ctypes.windll.kernel32
456             handle, mode = kernel.GetStdHandle(-11), ctypes.c_ulong()
457 
458             if not kernel.GetConsoleMode(handle, ctypes.byref(mode)) or not kernel.SetConsoleMode(handle,
                ↪  mode.value | 4):
459                 # Respect explicit color settings, terminal capability and the NO_COLOR environment flag.
460                 self.color = color == "always"
461 
462     def paint(self, text, code="36"):
463         """Wrap one fragment in ANSI codes; reset immediately to avoid color bleed."""
464 
465         # Reset ANSI styling after each fragment so red does not leak into neighboring values.
466         if self.color:
467             result_value = f'\x1b[{code}m{text}\x1b[0m'
468         else:
469             result_value = str(text)
470 
471         return result_value
472 
473     def section(self, title):
474         """Print a visible teaching boundary without affecting cryptographic data."""
```

```
475
476         print("\n" + self.paint("=" * 76, "36"))
477         print(self.paint(title, "1;36"))
478
479     def pause(self):
480         """Automatic execution remains the default unless --step was requested."""
481
482         # Pause only in interactive step mode; ordinary execution and tests remain automatic.
483         if self.step:
484             input("\nPress Enter to continue...")
485
486     def compare(self, correct, faulty):
487         """Compare two 16-byte states in AES row order and highlight fault effects."""
488
489         byte_values = []
490
491         for input_value, byte_2 in zip(correct, faulty):
492             byte_values.append(input_value ^ byte_2)
493
494         delta = bytes(byte_values)
495
496         print("          CLEAN STATE         FAULTY STATE        XOR DIFFERENCE")
497
498         for row in range(4):
499             clean_cells, faulty_cells, diff_cells = [], [], []
500
501             for column in range(4):
502                 # Render AES rows while reading bytes in their actual column-major order.
503                 index = 4 * column + row
504                 clean_cells.append(f"{correct[index]:02X}")
505
506                 if delta[index]:
507                     element_values_6 = self.paint(f'{faulty[index]:02X}', '31')
508                 else:
509                     element_values_6 = f'{faulty[index]:02X}'
510
511                 faulty_cells.append(element_values_6)
512
513                 if delta[index]:
514                     element_values_5 = self.paint(f'{delta[index]:02X}', '31')
515                 else:
516                     element_values_5 = '00'
517
518                 diff_cells.append(element_values_5)
519
520             print(f"row {row}:    {' '.join(clean_cells)}        {' '.join(faulty_cells)}        {'
                ↪  '.join(diff_cells)}")
521
522         changed = []
523
524         for candidate_index, byte_3 in enumerate(delta):
525             if byte_3:
526                 changed.append(candidate_index)
527
528         element_values = []
529
530         for byte in delta:
531             element_values.append(bin(byte).count('1'))
532
533         print(f'Changed bytes: {len(changed)}/16 | Changed bits: {sum(element_values)}/128')
534         print(f"Changed flat positions (4*column+row): {changed}")
```

F.2 Source excerpts: `sm4.py`

F.2.1 `observe`

Source lines 118–136.

```
118 def observe(observer, number, operation, value):
119     """Optional experiment callback. Normal cipher calls have no observer.
120
121     An observer may return a replacement value, or None to leave it unchanged.
122     Fault coordinates and masks belong to the external attack, never this file.
```

```
123     """
124
125     if observer is None:
126         return value
127
128     # Only the external observer can request a replacement; None preserves normal execution.
129     replacement = observer(number, operation, value)
130
131     if replacement is None:
132         result_value = value
133     else:
134         result_value = replacement
135
136     return result_value
```

F.2.2 round

Source lines 139–156.

```
139 def round(x: list[int], rk: int, observer=None, round_index=0):
140     """Return the next rolling window without modifying the supplied words.
141
142     A = X[i+1] XOR X[i+2] XOR X[i+3] XOR rk[i]
143     X[i+4] = X[i] XOR L(tau(A))
144     """
145
146     # Form the nonlinear input from the newest three words and the round key.
147     sbox_input = observe(observer, round_index, "Sbox input", x[1] ^ x[2] ^ x[3] ^ rk)
148
149     substituted = observe(observer, round_index, "Sbox output", sbox(sbox_input))
150
151     diffused = observe(observer, round_index, "Linear output", linear(substituted))
152
153     # XOR with the oldest word, then drop that word when returning the next window.
154     new_word = observe(observer, round_index, "New word", x[0] ^ diffused)
155
156     return x[1], x[2], x[3], new_word
```

F.2.3 generate_round_key

Source lines 159–167.

```
159 def generate_round_key(k: list[int], round: int) -> int:
160     """Compute K[i+4] from four consecutive schedule words and CK[i].
161
162     The parameter name round is retained for callers using keyword arguments.
163     Encryption and expansion share tau, but expansion must use linear_key.
164     """
165
166     # Use public CK for this schedule round; this is not an encryption round key input.
167     return k[0] ^ linear_key(sbox(k[1] ^ k[2] ^ k[3] ^ CK[round]))
```

F.2.4 key_expansion

Source lines 175–197.

```
175 def key_expansion(key: bytes) -> list[int]:
176     """Return rk0..rk31; each is one new word from the key recurrence."""
177
178     if len(key) != BLOCK_SIZE:
179         raise ValueError("SM4 key must contain exactly 16 bytes.")
180
181     master_words = struct.unpack(">4I", key)
182
183     # Whiten the master words with four fixed public FK constants before expansion.
184     schedule_words = []
185
186     for index in range(4):
187         schedule_words.append(master_words[index] ^ FK[index])
188
189
190     for i in range(N_ROUNDS):
```

```
191         # Keep previous words for readability; the recurrence consumes a window.
192         next_key = generate_round_key(schedule_words[i:i+4], i)
193 
194         schedule_words.append(next_key)
195 
196     # The initial four whitened words are not encryption round keys.
197     return schedule_words[4:]
```

F.2.5 sm4_process_block

Source lines 205–235.

```
205 def sm4_process_block(block: bytes, key: bytes, encrypt: bool, observer=None) -> bytes:
206     """Process a normal single block, without printing or built-in faults.
207 
208     The final four-word reversal is part of SM4, not a byte-endianness change.
209     Reversing only the key order is sufficient to run the inverse cipher.
210     """
211 
212     if len(block) != BLOCK_SIZE:
213         raise ValueError("SM4 block must contain exactly 16 bytes.")
214 
215     state = list(struct.unpack(">4I", block))
216 
217     round_keys = key_expansion(key)
218 
219     for i in range(N_ROUNDS):
220         # Decryption reuses the round function, changing only the round-key order.
221         if encrypt:
222             key_index = i
223         else:
224             key_index = N_ROUNDS - 1 - i
225 
226         state = list(observe(observer, i, "State input", tuple(state)))
227 
228         state = list(round(state, round_keys[key_index], observer, i))
229 
230         state = list(observe(observer, i, "State output", tuple(state)))
231 
232     # Reverse word order at output; do not reverse bytes within each word.
233     output = struct.pack(">4I", *state[::-1])
234 
235     return observe(observer, N_ROUNDS, "Ciphertext", output)
```

F.3 Source excerpts: attack_steps.py

F.3.1 sm4_observation

Source lines 219–369.

```
219 def sm4_observation(event, previous=None, known_bit=None):
220     """Explain every peeling operation, inverse-L contribution and candidate test."""
221 
222     number, byte = event['round'], event['byte']
223 
224     correct, faulty = bytes.fromhex(event['correct']), bytes.fromhex(event['faulty'])
225 
226     # Decode only keys already recovered at this point in the attack history.
227     keys = {}
228 
229     for row, candidate_value in event['peeled_with'].items():
230         keys[int(row)] = int(candidate_value, 16)
231 
232     for title, ciphertext in (("Correct", correct), ("Faulty", faulty)):
233         # Undo ciphertext reversal before evaluating the reverse round recurrence.
234         words = list(struct.unpack('>4I', ciphertext))[::-1]
235 
236         formatted_values_3 = []
237 
238         for word_2 in words:
239             formatted_values_3.append(f'{word_2:08X}')
240 
```

```
241         lines = [f'Ciphertext = {ciphertext.hex().upper()}', 'Undo ciphertext word reversal: ' + '
            ↪  '.join(formatted_values_3)]
242 
243         # Peel only rounds later than the target; each uses an already recovered subkey.
244         for r in range(31, number, -1):
245             value = words[0] ^ words[1] ^ words[2] ^ keys[r]
246             substituted = sm4.sbox(value)
247             diffused = sm4.linear(substituted)
248             restored = words[3] ^ diffused
249             lines += [f"Undo round {r}, using recovered rk{r}={keys[r]:08X}", f"A = {words[0]:08X} XOR
                ↪  {words[1]:08X} XOR {words[2]:08X} XOR {keys[r]:08X} = {value:08X}",
                ↪  f"tau(A)={substituted:08X}; L(tau(A))={diffused:08X}", f"Old word = {words[3]:08X} XOR
                ↪  {diffused:08X} = {restored:08X}"]
250             words = [restored, words[0], words[1], words[2]]
251             formatted_values_2 = []
252 
253             for word in words:
254                 formatted_values_2.append(f'word={word:08X}')
255 
256             lines.append('Restored chronological window: ' + ' '.join(formatted_values_2))
257 
258         if number == 31:
259             lines.append("No later round needs to be undone.")
260 
261         yield step(number, f"rk{number} byte {byte}: {title} round peeling", lines, correct, faulty)
262 
263     clean, fault = bytes.fromhex(event['clean_view']), bytes.fromhex(event['faulty_view'])
264 
265     c, f = struct.unpack('>4I', clean), struct.unpack('>4I', fault)
266 
267     delta_x = c[0] ^ f[0]
268 
269     lines = [f"DeltaX = {c[0]:08X} XOR {f[0]:08X} = {delta_x:08X}", "L is linear over GF(2). Apply inverse
        ↪  L to each set bit, then XOR contributions."]
270 
271     # Start the inverse-L result at zero and XOR each active input bit contribution.
272     accumulated = 0
273 
274     for bit in range(32):
275         basis = piret_sm4.inverse_linear(1 << bit)
276 
277         # An unset input bit contributes nothing; a set bit contributes its inverse basis image.
278         if delta_x & 1 << bit:
279             contribution = basis
280         else:
281             contribution = 0
282 
283         accumulated ^= contribution
284 
285         lines.append(f"bit {bit:02d}: input={(delta_x >> bit) & 1}; inverse basis={basis:08X};
            ↪  contribution={contribution:08X}; running XOR={accumulated:08X}")
286 
287     if accumulated != event['delta_b'] or sm4.linear(accumulated) != delta_x:
288         raise RuntimeError("Inverse-L lesson does not match the attack observation.")
289 
290     shift = 24 - byte * 8
291 
292     delta = (accumulated >> shift) & 255
293 
294     q = ((c[1] ^ c[2] ^ c[3]) >> shift) & 255
295 
296     lines += [f"DeltaB = {accumulated:08X}; L(DeltaB) = {sm4.linear(accumulated):08X} / CHECK PASS",
        ↪  f"Select big-endian byte {byte}: delta={delta:02X}", f"q = byte {byte} of ({c[1]:08X} XOR
        ↪  {c[2]:08X} XOR {c[3]:08X}) = {q:02X}"]
297 
298     yield step(number, f"rk{number} byte {byte}: inverse linear transform", lines, clean, fault)
299 
300     if known_bit is False:
301         allowed = (1, 2, 4, 8, 16, 32, 64, 128)
302     else:
303         allowed = (event['mask'],)
304 
305     fresh = set()
306 
```

```
307     lines = [f"Allowed errors = {values(allowed)}", "For every a=00..FF test S(a) XOR S(a XOR e) = delta.
        ↪  Then key byte = a XOR q."]
308
309     for value in range(256):
310         # Print every permitted error for this unknown input byte, including rejected tests.
311         tests = []
312
313         for error_2 in allowed:
314             tests.append((error_2, sm4.SBOX[value] ^ sm4.SBOX[value ^ error_2]))
315
316         has_match = False
317
318         for unused_index, result_2 in tests:
319             if result_2 == delta:
320                 has_match = True
321                 break
322
323         accepted = has_match
324
325         if accepted:
326             # Convert the compatible S-box input a to a round-key byte using public q.
327             fresh.add(value ^ q)
328
329         formatted_values = []
330
331         for error, result in tests:
332             formatted_values.append(f'e={error:02X}: {sm4.SBOX[value]:02X} XOR S[{value ^
                ↪  error:02X}]={sm4.SBOX[value]:02X} XOR {sm4.SBOX[value ^ error]:02X}={result:02X}')
333
334         calculations = '; '.join(formatted_values)
335
336         if accepted:
337             element_values_2 = 'ACCEPT'
338         else:
339             element_values_2 = 'REJECT'
340
341         lines.append(f'a={value:02X}; {calculations}; expected={delta:02X}; key={value ^ q:02X} / ' +
            ↪  element_values_2)
342
343     yield step(number, f"rk{number} byte {byte}: all 256 S-box inputs", lines, clean, fault)
344
345     # The display uses exactly the same set intersection as the attack solver.
346     if previous is None:
347         survivors = fresh
348     else:
349         survivors = previous & fresh
350
351     if sorted(survivors) != event['survivors']:
352         raise RuntimeError("Displayed SM4 candidate intersection disagrees with the solver.")
353
354     if previous is None:
355         lines_2 = '00..FF'
356     else:
357         lines_2 = values(previous)
358
359     if previous is None:
360         element_values = set(range(256))
361     else:
362         element_values = previous
363
364     lines = ['Fresh key bytes: ' + values(fresh), 'Previous: ' + lines_2, 'Removed: ' +
        ↪  values(element_values - fresh), 'Intersection: ' + values(survivors)]
365
366     if len(survivors) == 1:
367         lines.append(f"UNIQUE rk{number}[{byte}] = {next(iter(survivors)):02X}")
368
369     yield step(number, f"rk{number} byte {byte}: complete intersection", lines, clean, fault)
```

F.3.2 `sm4_schedule`

Source lines 431–487.

```
431 def sm4_schedule(keys):
432     """Reverse every SM4 schedule word, substitute each byte and remove FK."""
```

```
433
434     words_values = []
435
436     for row_2 in range(28, 32):
437         words_values.append(keys[row_2])
438
439     words = [None] * 32 + words_values
440
441     state = struct.pack('>4I', *words[32:])
442
443     element_values = []
444
445     for row in range(28, 32):
446         element_values.append(f'K[{row + 4}] = rk{row} = {keys[row]:08X}')
447
448     events = [step(31, 'Assemble SM4 recovered round keys', element_values, state)]
449
450     def record(index, recovered):
451         value = words[index+1] ^ words[index+2] ^ words[index+3] ^ sm4.CK[index]
452
453         substituted = sm4.sbox(value)
454
455         # Keep rotations within 32 bits because Python integers have arbitrary precision.
456         r13 = ((substituted << 13) | (substituted >> 19)) & 0xffffffff
457
458         r23 = ((substituted << 23) | (substituted >> 9)) & 0xffffffff
459
460         transformed = substituted ^ r13 ^ r23
461
462         lines = [f"U = {words[index+1]:08X} XOR {words[index+2]:08X} XOR {words[index+3]:08X} XOR
        ↪  CK[{index}]({sm4.CK[index]:08X}) = {value:08X}"]
463
464         computed_value_values = []
465
466         for candidate_index, byte in enumerate(value.to_bytes(4, 'big')):
467             computed_value_values.append(f'tau byte {candidate_index}: S[{byte:02X}]={sm4.SBOX[byte]:02X}')
468
469         lines += computed_value_values
470
471         lines += [f"B={substituted:08X}; rol13={r13:08X}; rol23={r23:08X}", f"L'(B) = {substituted:08X}
        ↪  XOR {r13:08X} XOR {r23:08X} = {transformed:08X}", f"K[{index}] = K[{index+4}] XOR L'(tau(U)) =
        ↪  {words[index+4]:08X} XOR {transformed:08X} = {recovered:08X}"]
472
473         words[index] = recovered
474
475         if index >= 4:
476             lines.append(f"DERIVED rk{index-4} = {recovered:08X}")
477
478         events.append(step(index, f"Reverse SM4 schedule: K[{index}]", lines, state))
479
480     master = piret_sm4.reverse_sm4(words[32:], record)
481
482     for i in range(4):
483         events.append(step(0, f"Remove FK[{i}]", [f"MK[{i}] = K[{i}] XOR FK[{i}] = {words[i]:08X} XOR
        ↪  {sm4.FK[i]:08X} = {words[i] ^ sm4.FK[i]:08X}"], master))
484
485     events.append(step(0, "Assemble SM4 master key", ["MK = MK[0] || MK[1] || MK[2] || MK[3]",
        ↪  master.hex().upper()], master))
486
487     return master, events
```

F.3.3 sm4_fault_trace

Source lines 490–541.

```
490 def sm4_fault_trace(clean_trace, fault_trace, number, byte, mask):
491     """Teacher-only propagation; these internal values never enter the solver."""
492
493     for r in range(number, 32):
494         clean, fault = clean_trace[r], fault_trace[r]
495
496         lines = ["TEACHER VIEW: target internals shown for explanation only."]
497
498         if r == number:
```

```
499             lines.append(f"Inject at round {r}, byte {byte}: A' = {clean['a']:08X} XOR {mask <<
                ↪  (24-8*byte):08X} = {fault['a']:08X}")
500
501         for label, field in (("S-box input", 'a'), ("S-box output", 'b'), ("Linear output", 'linear'),
            ↪  ("New word", 'new')):
502             lines.append(f"{label}: correct={clean[field]:08X}; faulty={fault[field]:08X};
                ↪  XOR={clean[field] ^ fault[field]:08X}")
503
504         for title, item in (("Correct", clean), ("Faulty", fault)):
505             computed_value_values = []
506
507             for index, value_2 in enumerate(item['a'].to_bytes(4, 'big')):
508                 computed_value_values.append(f'{title} S-box byte {index}:
                    ↪  S[{value_2:02X}]={sm4.SBOX[value_2]:02X}')
509
510             lines += computed_value_values
511             b = item['b']
512             rotations = []
513
514             for candidate_number in (2, 10, 18, 24):
515                 rotations.append((b << candidate_number | b >> 32 - candidate_number) & 4294967295)
516
517             formatted_values = []
518
519             for value in rotations:
520                 formatted_values.append(f'{value:08X}')
521
522             lines.append(f'{title} L: {b:08X} XOR ' + ' XOR '.join(formatted_values) + f" =
                ↪  {item['linear']:08X}")
523
524         # These display rows are A, tau(A), L(tau(A)), and the new word, not a rolling cipher state.
525         element_values_2 = []
526
527         for candidate_field_2 in ('a', 'b', 'linear', 'new'):
528             element_values_2.append(clean[candidate_field_2])
529
530         clean_state = struct.pack('>4I', *element_values_2)
531
532         element_values = []
533
534         for candidate_field in ('a', 'b', 'linear', 'new'):
535             element_values.append(fault[candidate_field])
536
537         fault_state = struct.pack('>4I', *element_values)
538
539         lines.append("Displayed rows: A, tau(A), L(tau(A)), new word.")
540
541         yield step(r, f"Fault propagation: SM4 round {r}", lines, clean_state, fault_state)
```

F.3.4 `verification`

Source lines 544–565.

```
544 def verification(algorithm, plaintext, key, expected):
545     """Check the recovered key by both forward and inverse block operations."""
546
547     if algorithm == 'aes':
548         encrypt = aes.aes_encrypt
549     else:
550         encrypt = sm4.sm4_encrypt_block
551
552     if algorithm == 'aes':
553         decrypt = aes.aes_decrypt
554     else:
555         decrypt = sm4.sm4_decrypt_block
556
557     # Validate recovered key material through both forward and inverse block operations.
558     actual = encrypt(plaintext, key)
559
560     recovered = decrypt(expected, key)
561
562     if actual != expected or recovered != plaintext:
563         raise RuntimeError('Recovered key failed encryption/decryption verification.')
```

```
564
565     return [step(0, "Verify recovered key: encrypt and decrypt", [f"Recovered key = {key.hex().upper()}",
        ↪  f"P = {plaintext.hex().upper()}", f"Expected C = {expected.hex().upper()}", f"Encrypt(P, recovered
        ↪  key) = {actual.hex().upper()} / PASS", f"Decrypt(C, recovered key) = {recovered.hex().upper()} /
        ↪  PASS", "This verifies the observed pair; simulation ground-truth equality is checked
        ↪  separately."], expected)]
```

F.3.5 `emit`

Source lines 568–585.

```
568 def emit(terminal, events):
569     """Print the same timeline as the GUI; preserve red changed bytes in logs."""
570
571     for event in events:
572         terminal.section(event['title'])
573
574         correct, faulty = bytes.fromhex(event['reference']), bytes.fromhex(event['state'])
575
576         # Color changes against the paired clean state; do not color all bytes indiscriminately.
577         if correct != faulty:
578             if hasattr(terminal, 'pair'):
579                 terminal.pair(correct, faulty)
580             else:
581                 terminal.compare(correct, faulty)
582
583         print(event['detail'])
584
585         terminal.pause()
```

F.4 Source excerpts: `terminal_log.py`

F.4.1 `Tee`

Source lines 85–108.

```
85 class Tee:
86     """Keep the terminal's color detection and preserve its exact output."""
87     def __init__(self, terminal, output):
88         self.terminal = terminal
89
90         self.output = output
91
92     def write(self, text):
93         # Mirror each write to both destinations without removing ANSI color information.
94         self.terminal.write(text)
95
96         self.output.write(text)
97
98         self.output.flush()
99
100        return len(text)
101
102    def flush(self):
103        self.terminal.flush()
104
105        self.output.flush()
106
107    def __getattr__(self, name):
108        return getattr(self.terminal, name)
```

F.4.2 `terminal_log`

Source lines 111–144.

```
111 @contextmanager
112 def terminal_log(name, folder=None):
113     """Capture stdout and stderr; importing a cipher never creates a log."""
114
115     if folder is None:
116         directory = LOG_FOLDER
```

```
117     else:
118         directory = Path(folder)
119 
120     directory.mkdir(parents=True, exist_ok=True)
121 
122     # Remove only this program's old timestamped logs from the previous format.
123     legacy_name = re.compile(re.escape(name) + r"_\d{8}T\d{6}_\d{6}Z\.(txt|html)")
124 
125     for previous in directory.iterdir():
126         if previous.is_file() and legacy_name.fullmatch(previous.name):
127             previous.unlink()
128 
129     path = directory / f"{name}.txt"
130 
131     path.with_suffix(".html").write_text(colored_html(""), encoding="utf-8")
132 
133     # Opening in write mode replaces this program's previous run rather than appending.
134     with path.open("w", encoding="utf-8") as output:
135         try:
136             # Capture stderr too, including tracebacks, while preserving live terminal output.
137             with redirect_stdout(Tee(sys.stdout, output)), redirect_stderr(Tee(sys.stderr, output)):
138                 print(f"Log file: {path}")
139                 print(f"Colored HTML log: {path.with_suffix('.html')}")
140                 yield path
141         # Finalize the browser-readable HTML even after an exception or interruption.
142         finally:
143             output.flush()
144             path.with_suffix(".html").write_text(colored_html(path.read_text(encoding="utf-8")),
                ↪  encoding="utf-8")
```

F.4.3 `run_with_log`

Source lines 147–169.

```
147 def run_with_log(main, name):
148     """Save normal output, interrupted lessons and unexpected error details."""
149 
150     with terminal_log(name):
151         try:
152             # Keep logging active throughout the executable's main function, including a GUI session.
153             code = main()
154         # Treat Ctrl+C and a closed input stream as an interrupted lesson, with a recorded exit code.
155         except (KeyboardInterrupt, EOFError):
156             print("\nLesson stopped.")
157             code = 130
158         except Exception:
159             traceback.print_exc()
160             code = 1
161 
162         if code is None:
163             code = 0
164         else:
165             code = code
166 
167         print(f"Exit status: {code}")
168 
169         return code
```